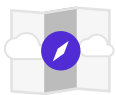


.NET Aspire documentation

Learn about .NET Aspire, an opinionated, cloud ready stack for building observable, production ready, distributed applications. Browse API reference, sample code, tutorials, quickstarts, conceptual articles and more.



OVERVIEW
[.NET Aspire overview](#)



QUICKSTART
[Build your first .NET Aspire solution](#)



CONCEPT
[App host orchestration](#)



CONCEPT
[.NET Aspire integrations](#)



CONCEPT
[Service discovery](#)



DEPLOY
[Deploy a .NET Aspire projects](#)



OVERVIEW
[.NET Aspire dashboard](#)



WHAT'S NEW
[What's new in .NET Aspire](#)

Explore various aspects of .NET Aspire, including getting started, storage, database, messaging, caching, frameworks, deployment, and troubleshooting.

Get Started

-  [.NET Aspire overview](#)
-  [Build your first .NET Aspire solution](#)
-  [Add a Node.js app to a .NET Aspire project](#)

Storage integrations

-  [Connect to storage with .NET Aspire](#)
-  [Azure Blob Storage](#)
-  [Azure Storage Queues](#)
-  [Azure Table Storage](#)

Database integrations

-  [Connect to SQL Server with EF Core](#)
-  [PostgreSQL database](#)
-  [PostgreSQL with EF Core](#)
-  [Azure Cosmos DB](#)

-  [.NET Aspire setup and tooling](#)
-  [.NET Aspire service discovery](#)
-  [.NET Aspire and launch profiles](#)
-  [.NET Aspire service defaults](#)
-  [.NET Aspire dashboard](#)
-  [.NET Aspire dashboard and GitHub Copilot](#)
-  [.NET Aspire local networking](#)

-  [Persist volume mount sample](#)

-  [Azure Cosmos DB with EF Core](#)
-  [SQL Database](#)
-  [SQL Database with EF Core](#)
-  [Entity Framework Core migrations](#)
-  [MySQLConnector Database](#)
-  [MongoDB Database](#)

Messaging integrations

-  [Implement Messaging with .NET Aspire](#)
-  [Azure Event Hubs](#)
-  [Azure Service Bus](#)
-  [Azure Web PubSub](#)
-  [RabbitMQ](#)
-  [Apache Kafka](#)
-  [NATs](#)

Caching integrations

-  [Improve app caching with .NET Aspire](#)
-  [Stack Exchange Redis caching overview](#)
-  [Redis caching](#)
-  [Redis output caching](#)
-  [Redis distributed caching](#)

Framework integrations

-  [Use Orleans with .NET Aspire](#)
-  [Orleans voting sample](#)
-  [Use Dapr with .NET Aspire](#)
-  [Dapr integration sample !\[\]\(07221912d1bf206beb97cefd77af5f78_img.jpg\)](#)

Deployment

-  [Overview](#)
-  [Deploy to Azure Container Apps](#)
-  [Deploy using the Azure Developer CLI](#)
-  [Integrate with Application Insights](#)
-  [.NET Aspire deployment manifest format](#)

Troubleshooting

-  [Allow unsecure transport](#)
-  [Unable to install workload](#)
-  [Untrusted localhost certificate](#)
-  [The specified name is already in use](#)
-  [Container runtime appears to be unhealthy](#)
-  [The connection string is missing](#)
-  [Ask questions on Discord !\[\]\(7a98cad912aaae353c9dca67332b4d62_img.jpg\)](#)
-  [Stack Overflow — .NET Aspire !\[\]\(8f263363c1e0b987d1237a323eb0b143_img.jpg\)](#)

Training

-  [Introduction to .NET Aspire](#)
-  [Create a .NET Aspire project](#)
-  [Use telemetry in a .NET Aspire project](#)
-  [Use databases in a .NET Aspire project](#)
-  [Improve performance with a cache in a .NET Aspire project](#)
-  [Send messages with RabbitMQ in a .NET Aspire project](#)

.NET extensions

Learn about .NET extensions, including logging, dependency injection, configuration, and more. All of which are fundamental in .NET Aspire.

Fundamentals

- [Logging](#)
- [Dependency injection](#)
- [Configuration](#)
- [Make HTTP requests](#)

Telemetry

- [.NET observability with OpenTelemetry](#)
- [Networking telemetry](#)
- [.NET SDK telemetry](#)

Resiliency

- [Introduction to resilient app dev](#)
- [Build resilient HTTP apps](#)
- [Implement resiliency in a cloud-native ASP.NET Core microservice](#)

Observability

- [.NET app health checks in C#](#)
- [App health checks in ASP.NET Core](#)
- [Diagnostic tools in .NET](#)
- [Diagnostic resource monitoring in .NET](#)

.NET community resources

Find community resources for .NET, including webcasts, shows, open-source projects, and more.

.NET

- [.NET documentation](#)
- [.NET Aspire samples browser](#)
- [ASP.NET documentation](#)
- [Azure documentation](#)
- [C# documentation](#)
- [.NET Discord !\[\]\(b792654f2cef9719eabeb6c5be00811e_img.jpg\)](#)
- [Official .NET Aspire Collection](#)

Webcasts and shows

- [Azure Friday](#)
- [The Cloud Native Show](#)
- [On .NET](#)
- [On .NET Live !\[\]\(2bae76de5ebbd5c4d7d47162f1673734_img.jpg\)](#)
- [.NET Community Standup !\[\]\(c8d5e2ee847d658b2a7eb88bf9ce84ac_img.jpg\)](#)
- [Welcome to .NET Aspire !\[\]\(e8c3b4b3458ce63d765e2b60d287abf5_img.jpg\)](#)

Open source

- [.NET Aspire !\[\]\(84f47badaad7772cd95667a7c387a639_img.jpg\)](#)
- [.NET Aspire samples !\[\]\(ab1cd3423001ff994d2c02189fd012b0_img.jpg\)](#)
- [.NET samples !\[\]\(866b710dee4cab3cb18eb5fb3c5660b5_img.jpg\)](#)
- [.NET Platform !\[\]\(55c726ebb9ccb06bb22589d9bcf71bf8_img.jpg\)](#)
- [.NET Runtime !\[\]\(713ead65ca77ea070ece03d919f9bcf5_img.jpg\)](#)
- [ASP.NET Core !\[\]\(2c5214f6585febee00dda3231bb7933b_img.jpg\)](#)

Community

[Follow @dotnet on X](#) ↗

[Follow @dotnet on Mastodon](#) ↗

[.NET Foundation](#) ↗

[We Are .NET](#) ↗

[.NET Aspire YouTube playlist](#) ↗

[.NET Aspire on YouTube](#) ↗

API and language reference

Search the .NET API and language reference documentation.

[.NET Aspire API reference](#)

API reference documentation
for .NET Aspire

[.NET API reference](#)

API reference documentation
for .NET

[ASP.NET Core API reference](#)

API reference documentation
for ASP.NET Core

Are you interested in contributing to the .NET Aspire docs? For more information, see our [contributor guide](#).
Interested in the official support policy, see [.NET Aspire Support Policy](#) ↗.

.NET Aspire — what's new?

Welcome to what's new in .NET Aspire docs. Use this page to quickly find the latest changes.

.NET Aspire release documentation

WHAT'S NEW

[What's new in .NET Aspire 9.3](#)

[What's new in .NET Aspire 9.2](#)

[What's new in .NET Aspire 9.1](#)

[What's new in .NET Aspire 9.0](#)

[Announcing .NET Aspire 8.2](#) 

[What's new in .NET Aspire 8.1](#) 

[.NET Aspire 8.0 \(General Availability\)](#) 

.NET Aspire GitHub release notes

WHAT'S NEW

[.NET Aspire 9.2](#) 

[.NET Aspire 9.1](#) 

[.NET Aspire 9.0](#) 

[.NET Aspire 8.2.2](#) 

[.NET Aspire 8.2.1](#) 

[.NET Aspire 8.2.0](#) 

[.NET Aspire 8.1.0](#) 

[.NET Aspire 8.0.2](#) 

[.NET Aspire 8.0.0](#) 

Latest documentation updates

WHAT'S NEW

[June 2025](#)

[May 2025](#)

[April 2025](#)

Find language updates

WHAT'S NEW

[What's new in C#](#)

[What's new in F#](#)

[What's new for Visual Basic](#)

Get involved - contribute

OVERVIEW

[.NET Aspire docs repository](#) 

[Project structure and labels for issues and pull requests](#)

[.NET Foundation](#) 

CONCEPT

[Microsoft docs contributor guide](#)

[.NET docs contributor guide](#)

.NET developers

DOWNLOAD

[Download the .NET SDK](#) 

SAMPLE

[.NET Aspire samples browser](#)



GET STARTED

[.NET on Q&A](#)

[.NET tech community forums](#) [↗](#)



WHAT'S NEW

[Community](#) [↗](#)

Related what's new pages



WHAT'S NEW

[.NET docs updates](#)

[ASP.NET Core docs updates](#)

[Xamarin docs updates](#)

[.NET release notes](#) [↗](#)

[ASP.NET Core release notes](#)

[C# compiler \(Roslyn\) release notes](#) [↗](#)

[Visual Studio release notes](#)

[Visual Studio Code release notes](#) [↗](#)

What's new in .NET Aspire 9.3

Article • 05/19/2025

 .NET Aspire 9.3 is the next minor version release of .NET Aspire. It supports:

- .NET 8.0 Long Term Support (LTS)
- .NET 9.0 Standard Term Support (STS)

If you have feedback, questions, or want to contribute to .NET Aspire, collaborate with us on [GitHub](#) or join us on [Discord](#) to chat with team members.

It's important to note that .NET Aspire releases out-of-band from .NET releases. While major versions of .NET Aspire align with major .NET versions, minor versions are released more frequently. For more information on .NET and .NET Aspire version support, see:

- [.NET support policy](#): Definitions for LTS and STS.
- [.NET Aspire support policy](#): Important unique product life cycle details.

App model enhancements

Zero-friction container configuration

Many container integrations now expose **first-class helpers** to set ports, usernames, and passwords without digging through internal properties. All three settings can be supplied **securely via parameters**, keeping secrets out of source:

C#

```
var pgPwd = builder.AddParameter("pg-pwd", secret: true);

builder.AddPostgres("pg")
    .WithHostPort(6045)           // choose the host-side port
    .WithPassword(pgPwd)         // reference a secret parameter
```

The new `WithHostPort`, `WithPassword`, and `WithUserName` (or equivalent per-service) extension methods are available on **PostgreSQL**, **SQL Server**, **Redis**, and several other container resources, giving you consistent, declarative control across the stack.

Streamlined custom URLs

9.3 makes resource links both **smarter** and **easier** to place:

- **Pick where a link appears** – each link now carries a `UrlDisplayLocation` (`SummaryAndDetails` or `DetailsOnly`), so you can keep diagnostic links out of the main grid yet still see them in the details pane.
- **Relative paths are auto-resolved** – hand the helper `"/health"` and Aspire rewrites it to the full host-qualified URL when the endpoint is allocated.
- **Multiple links per endpoint** – an overload of `WithUrlForEndpoint` lets you attach extra URLs (docs, admin UIs, probes) to the same endpoint without redefining it.
- **Endpoint helper inside callbacks** – `context.GetEndpoint("https")` fetches the fully-resolved endpoint so you can build custom links programmatically.
- **Custom URLs for any resource** – `WithUrl*` also works for custom resources.

C#

```
var frontend = builder.AddProject<Projects.Frontend>("frontend")

// Hide the plain-HTTP link from the Resources grid
.WithUrlForEndpoint("http",
    url => url.DisplayLocation = UrlDisplayLocation.DetailsOnly)

// Add an extra link under the HTTPS endpoint that points to /health
.WithUrlForEndpoint("https", ep => new()
{
    Url = "/health", // relative path supported
    DisplayText = "Health",
    DisplayLocation = UrlDisplayLocation.DetailsOnly
});
```

With these tweaks you can further customize your local dev stack by surfacing the right links in the right place.



Hide resources without "faking" their state

Historically the only way to keep a resource out of the Dashboard was to put it in the `Hidden` state—a hack that also made the resource look "terminal" to APIs such as `WaitForResourceAsync`. In 9.3 every snapshot now carries a boolean `IsHidden` flag, completely decoupling *visibility* from *lifecycle state*.

- **Cleaner defaults** – low-level helpers like `AddParameter` and `AddConnectionString` mark themselves hidden so they don't clutter the UI:

C#

```
var apiKey = builder.AddParameter("api-key", secret: true); // IsHidden = true ✓
```

- **Accurate waits & health flows** – `WaitForResourceAsync` was updated to treat `IsHidden` as a separate predicate, so hidden resources can still be awaited or surfaced programmatically without special-casing states.

This small change removes ambiguity in the model while giving you precise control over what shows up in the Dashboard.

New lifecycle events

.NET Aspire 9.3 introduces two new lifecycle events that make it easier to build custom resources with predictable behavior—without relying on hacks like `Task.Run` or polling:

`InitializeResourceEvent`

This event fires **after a resource is added**, but **before endpoints are allocated**. It's especially useful for custom resources that don't have a built-in lifecycle (like containers or executables), giving you a clean place to kick off background logic, set default state, or wire up behavior.

For example, this minimal custom resource publishes a running state when initialized:

C#

```
var myCustom = new MyCustomResource("my-resource");

builder.AddResource(myCustom);
builder.Eventing.Subscribe<InitializeResourceEvent>(myCustom, async (e, ct) =>
{
    await e.Notifications.PublishUpdateAsync(e.Resource,
        s => s with { State = KnownResourceStates.Running });
});
```

This replaces awkward patterns like `Task.Run` inside constructors or `Configure()` methods. You can see a more complex version in the [TalkingClock sample](#) in the official Aspire samples repo.

`ResourceEndpointsAllocatedEvent`

This event fires once a resource's endpoints have been assigned (e.g., after port resolution or container allocation). It's scoped per resource, so you can safely get an `EndpointReference` and build derived URLs or diagnostics.

C#

```
builder.Eventing.Subscribe<ResourceEndpointsAllocatedEvent>((e, ct) =>
{
    if (e.Resource is IResourceWithEndpoints resource)
    {
        var http = resource.GetEndpoint("http");

        Console.WriteLine($"Endpoint http - Allocated {http.IsAllocated}, Port:
{http.Port}");
    }

    return Task.CompletedTask;
});
```

These events make resource authoring smoother, safer, and more deterministic—no lifecycle guesswork needed.

YARP Integration (Preview)

.NET Aspire 9.3 introduces **preview support for YARP** [\(Yet Another Reverse Proxy\)](#)—a long-requested addition that brings reverse proxying into the Aspire application model.

This integration makes it easy to add a lightweight proxy container to your distributed app, powered by the official [YARP container image](#). It currently supports **configuration-based routing only**, using a JSON file you supply.

Add a reverse proxy to your Aspire app:

C#

```
builder.AddYarp("apigateway")
    .WithConfigFile("yarp.json")
    .WithReference(basketService)
    .WithReference(catalogService);
```

The config file is mounted into the container and used as the runtime YARP configuration.

Example *yarp.json*:

JSON

```
{
  "ReverseProxy": {
    "Routes": {
      "catalog": {
        "ClusterId": "catalog",
```

```

    "Match": {
      "Path": "/catalog/{**catch-all}"
    }
  },
  "basket": {
    "ClusterId": "basket",
    "Match": {
      "Path": "/basket/{**catch-all}"
    }
  }
},
"Clusters": {
  "catalog": {
    "Destinations": {
      "catalog/d1": {
        "Address": "http://catalog/"
      }
    }
  },
  "basket": {
    "Destinations": {
      "basket/d1": {
        "Address": "http://basket/"
      }
    }
  }
}
}
}
}

```

The `.WithReference(...)` calls automatically ensure that the proxy container can resolve the referenced services by name (`catalog`, `basket`), using Aspire's internal network graph.

⚠️ Known limitations in this preview

- **Only configuration-based routing is supported.** Code-based or programmatic route generation is not available yet.
- **The configuration file is not deployed** as part of publish operations—you must manage the file manually.
- **Routing from containers to projects will not work on Podman**, due to host-to-container networking limitations.

💡 Tip

💡 Want to learn more about authoring YARP configs? See the official [YARP documentation](#). 🛠️ This integration is in preview—APIs and behavior may evolve. Feedback welcome!

MySQL `AddDatabase` now creates the database

In .NET Aspire 9.3, the MySQL integration now supports **automatic database creation** via the `AddDatabase` API—matching the behavior already available for SQL Server and PostgreSQL.

Previously, calling `.AddDatabase("dbname")` on a MySQL resource only created a logical reference in Aspire's app model—it did **not** create the database on the server. This mismatch caused confusion, especially when users expected Aspire to provision the database like it does for other integrations.

New behavior in 9.3:

C#

```
var mysql = builder.AddMySQL("db");  
  
mysql.AddDatabase("appdb");
```

At runtime, Aspire now executes a `CREATE DATABASE` command for `"appdb"` against the running MySQL container or server. If the database already exists, the command is skipped safely.

This brings MySQL in line with the broader Aspire database ecosystem:

 Expand table

Integration	Automatically creates database?
SQL Server	 Yes
PostgreSQL	 Yes
MySQL	 Yes (new in 9.3)
MongoDB	 No (not needed; created on first write)
Oracle	 No (not supported yet)

No additional configuration is required—the same `AddDatabase` call you already use now provisions the database for you behind the scenes.

Dashboard delights

GitHub Copilot in the dashboard

Introducing GitHub Copilot in the .NET Aspire dashboard! GitHub Copilot is your new AI debugging assistant.

GitHub Copilot supercharges the dashboard's OpenTelemetry debugging and diagnostics experience. With AI, you can:

- Review hundreds of log messages with a single click
- Investigate the root cause of errors across multiple apps
- Highlight performance issues in traces
- Explain obscure error codes using AI's huge knowledge repository

You'll have access to Copilot in the dashboard when you launch your app from VS Code or Visual Studio.

The screenshot shows the .NET Aspire dashboard in a web browser. The main view is 'Resources', which displays a table of running services. The table has columns for Name, State, URLs, and Actions. The services listed are: apigateway, basketcache, messaging, pgadmin, postgres, catalogdb, rediscommander, redisinsight, aspire-dashboard, basketservice, catalogdbapp, and catalogservice-awefko. All services are in a 'Running' state. To the right of the table is a sidebar with a search filter and a 'Filter...' button. On the far right, there is a 'GitHub Copilot chat' panel. This panel contains a search bar, a 'Summarize resources' button, a 'Recent activity' button, and a 'Help' button. Below these buttons, there is a message from Copilot: 'Copilot uses AI. Check for mistakes. Aspire uses your Visual Studio connection to generate Copilot insights.' At the bottom of the chat panel is a text input field labeled 'Type your message...' and a 'GPT-4.1' dropdown menu.

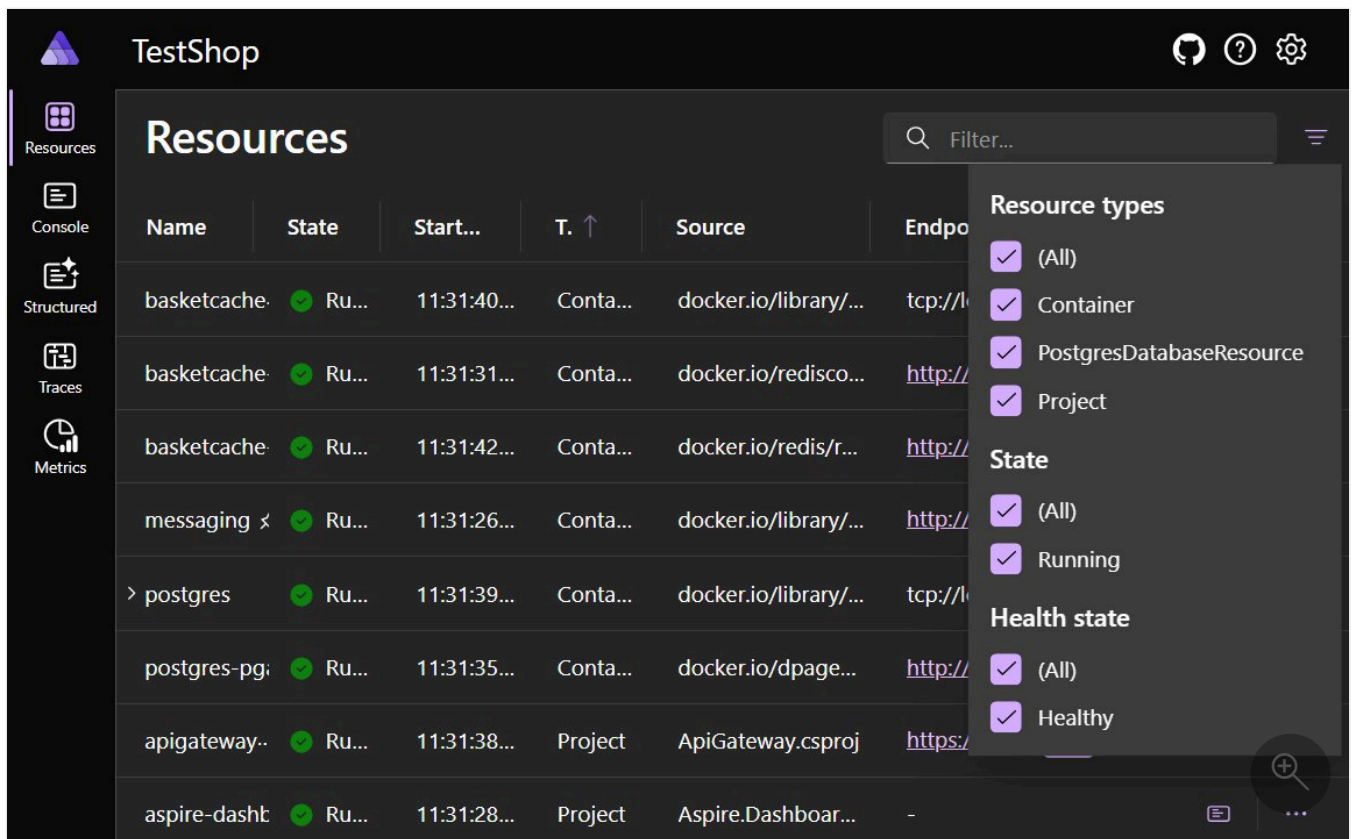
Name	State	URLs	Actions
apigateway	Running	http://localhost:53035	...
basketcache	Running	tcp://localhost:53034	...
messaging	Running	http://localhost:53033 +1	...
pgadmin	Running	PG Admin	...
postgres	Running	tcp://localhost:53031	...
catalogdb	Running	-	...
rediscommander	Running	Redis Commander	...
redisinsight	Running	Redis Insight	...
aspire-dashboard	Running	-	...
basketservice	Running	https://localhost:5305 +1	...
catalogdbapp	Running	https://localhost:7163/re... +1	...
catalogservice-awefko	Running	https://localhost:5239/sw... +1	...

For more details on requirements and how to get started, see [GitHub Copilot in the .NET Aspire dashboard](#).

Remembers your filter settings

The .NET Aspire dashboard now **remembers your resource filter settings** between sessions. Previously, if you filtered the Resources view (for example, to hide support services or highlight only frontend apps), those filters were reset on page reload.

As of 9.3, filter state is **persisted in local storage**, so your selections stick across refreshes and restarts. This small improvement makes it easier to focus on the parts of your app that matter most—especially in large graphs with many supporting services like Redis, SQL, or queues.



The screenshot shows the TestShop dashboard with the 'Resources' view selected. A filter menu is open, showing options for Resource types, State, and Health state. The Resources table lists various services like basketcache, messaging, postgres, and aspire-dash.

Name	State	Start...	T. ↑	Source	Endpo
basketcache	✓ Ru...	11:31:40...	Conta...	docker.io/library/...	tcp://l...
basketcache	✓ Ru...	11:31:31...	Conta...	docker.io/redisco...	http://...
basketcache	✓ Ru...	11:31:42...	Conta...	docker.io/redis/r...	http://...
messaging ✕	✓ Ru...	11:31:26...	Conta...	docker.io/library/...	http://...
> postgres	✓ Ru...	11:31:39...	Conta...	docker.io/library/...	tcp://l...
postgres-pgi	✓ Ru...	11:31:35...	Conta...	docker.io/dpage...	http://...
apigateway..	✓ Ru...	11:31:38...	Project	ApiGateway.csproj	https://...
aspire-dash	✓ Ru...	11:31:28...	Project	Aspire.Dashboar...	-

Resource types

- ☒ (All)
- ☒ Container
- ☒ PostgresDatabaseResource
- ☒ Project

State

- ☒ (All)
- ☒ Running

Health state

- ☒ (All)
- ☒ Healthy

Uninstrumented resources now appear in Traces

In 9.3, the dashboard can now **visualize outgoing calls to resources that don't emit their own telemetry**—such as databases, caches, and other infrastructure components that lack built-in tracing.

Previously, these dependencies were invisible in the **Traces** view unless they were emitting OTLP traces. Now, if your app makes an HTTP, SQL, or Redis call to a **modeled Aspire resource** that doesn't emit spans itself, Aspire still shows it as a **referenced peer** in the trace timeline.

This helps you:

- Understand the full chain of dependencies—even if some components are passive
- Debug latency or failures in calls to uninstrumented services
- Keep the trace UI consistent across infrastructure types

Important

 This is especially useful for services like SQL Server, PostgreSQL, Redis, or blob storage where outgoing client telemetry exists, but the service itself doesn't participate in

distributed tracing.

 No instrumentation changes are needed—Aspire infers the mapping based on resource references.

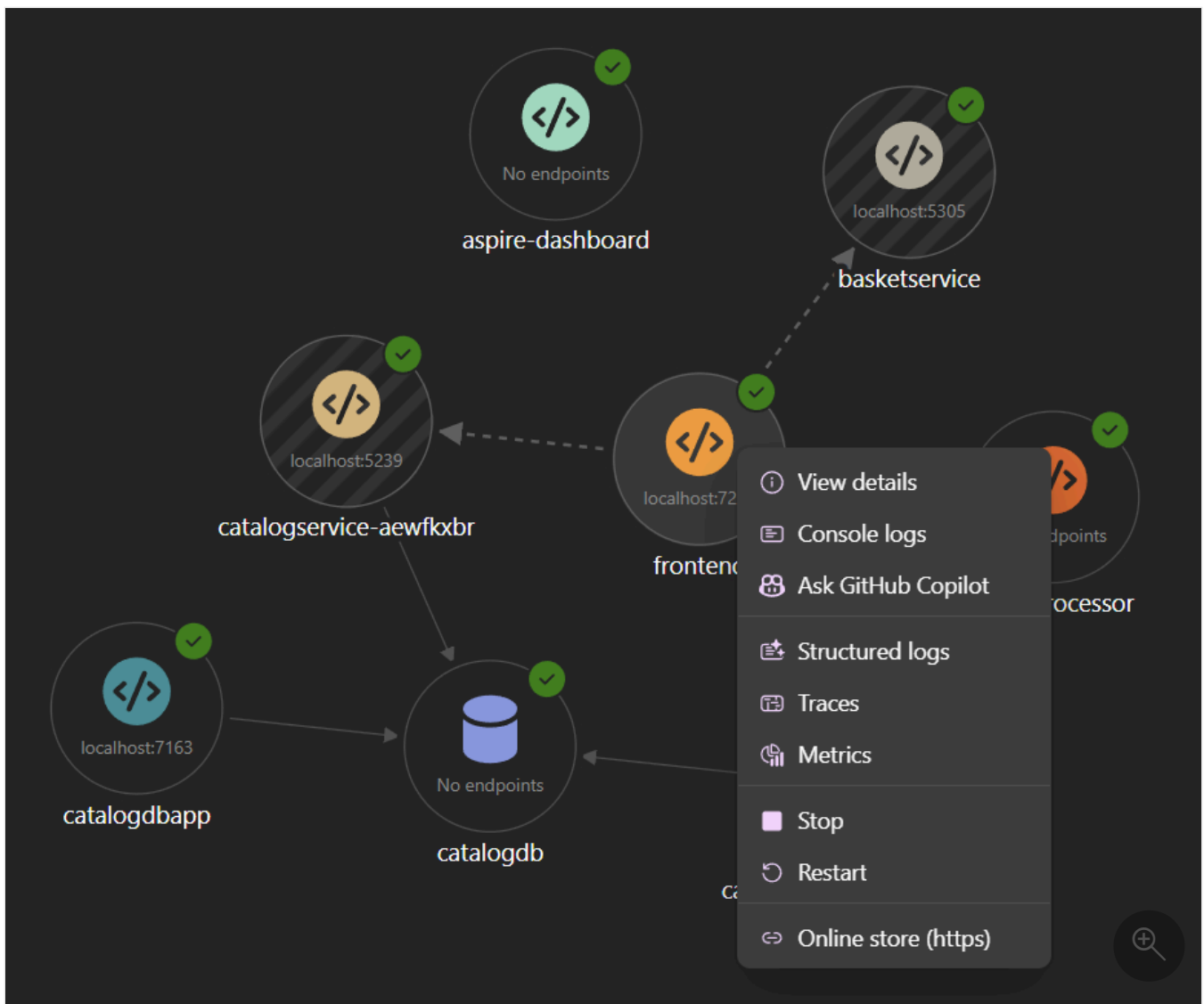
Traces

Resource: postgres 00 🗑️

Timestamp	Name	Spans
1:35:07.634 pm	catalogdbapp: Initializing catalog database c0f51b4	catalogdbapp (9) postgres (8)
1:35:43.455 pm	frontend: GET / 00f10d5	frontend (5) catalogservice-aewflobr (2) basketservice (2) basketcache (1) postgres (1)
1:35:45.348 pm	frontend: GET /catalog/images/{id} 69bf34e	frontend (2) catalogservice-hdcryfx (2) postgres (1)
1:35:45.348 pm	frontend: GET /catalog/images/{id} 7340931	frontend (2) catalogservice-hdcryfx (2) postgres (1)
1:35:45.348 pm	frontend: GET /catalog/images/{id} 7465ac1	frontend (2) catalogservice-hdcryfx (2) postgres (1)
1:35:46.989 pm	frontend: GET /catalog/images/{id} b113367	frontend (2) catalogservice-hdcryfx (2) postgres (1)
1:35:46.989 pm	frontend: GET /catalog/images/{id} afa0372	frontend (2) catalogservice-hdcryfx (2) postgres (1)

Resource context menus & quick-launch actions

.NET Aspire 9.3 makes the dashboard more interactive and easier to navigate by introducing new **context menus** and enhancing how **resource URLs** are surfaced across views.



🕒 Right-click context menus in the graph

You can now **right-click** any resource node in the **Resource Graph** view to bring up a context menu with quick actions:

- Open structured logs, console logs, traces, or metrics for that resource
- Launch external URLs associated with the resource (like PGAdmin, Swagger, or Grafana)
- Jump directly to the resource's detail pane

This reduces the number of clicks and lets you stay in the graph while investigating specific services.

🔗 Resource URLs in console log actions

Resource URLs defined via `WithUrlForEndpoint(...)` are now **more prominently integrated** into the dashboard UI. They appear:

- In the **console logs view** action bar

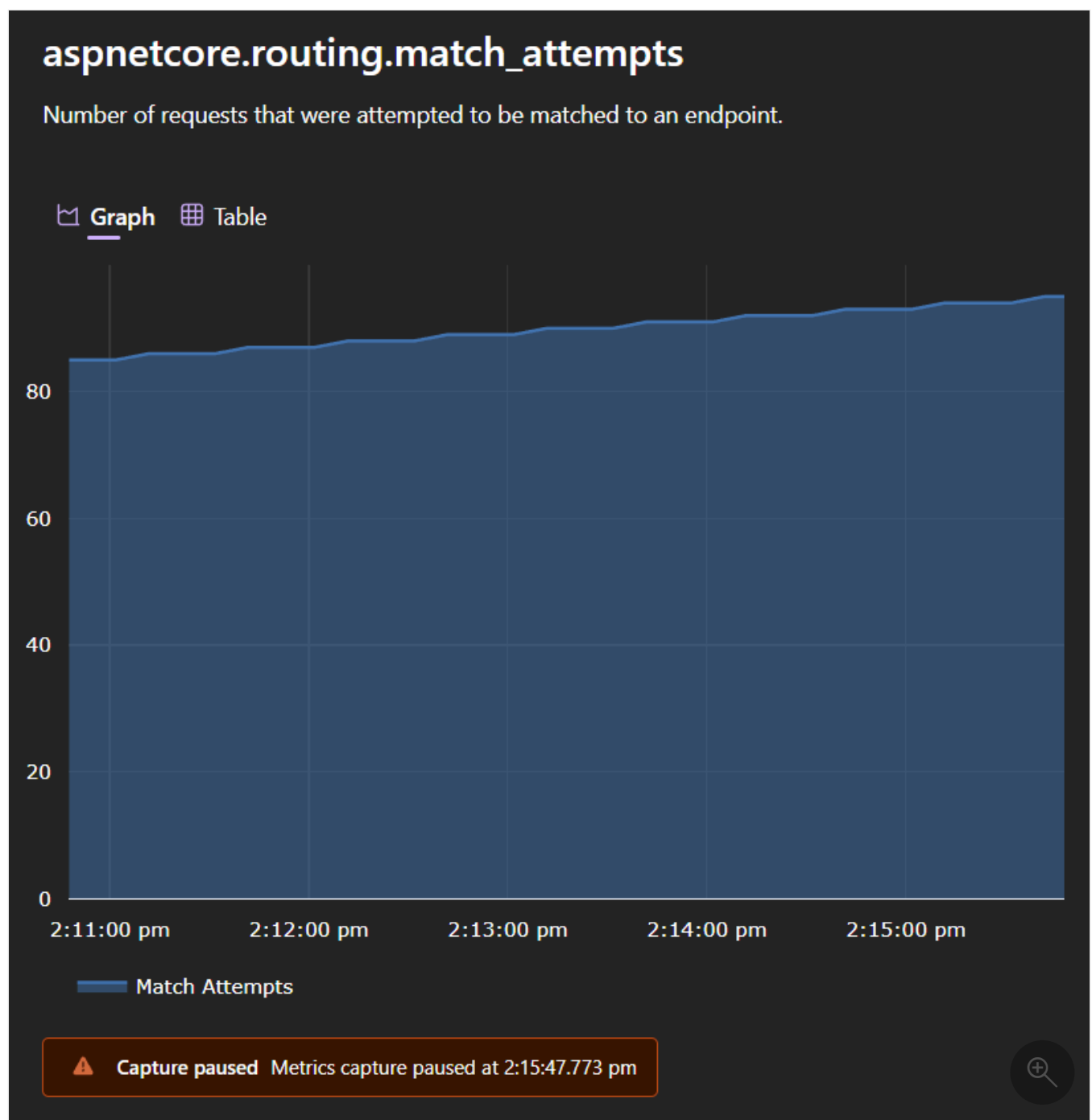
- In the new **right-click menus**
- On the **resource detail pane**, as before

This makes common destinations—like admin UIs, health checks, and docs—instantly accessible wherever you're working.

Together, these improvements turn the Aspire dashboard into a true control plane for navigating your distributed app—**less friction, more focus**.

II Metrics pause warning

The dashboard now shows a **warning banner** when metrics collection is paused. This makes it clear that data may be stale if you've temporarily halted telemetry.



Friendly names in console logs

When a resource has only **one replica**, the Aspire dashboard now uses the **friendly resource name** (like `frontend`, `apigateway`, or `redis`) instead of the replica ID (like `frontend-0`) in the **console logs view**.

This small change makes logs easier to read and reduces visual noise—especially in common single-instance setups during development.

Note

In multi-replica scenarios, Aspire still uses full replica IDs so you can distinguish between instances.

Deployment & publish

Improvements to in-preview publisher model & compute environment support

In 9.2, we shipped our first iteration of "publishers", a flexible way to configure deployments to any cloud in the app host. To ensure more flexibility, .NET Aspire 9.3 includes a **new and improved** publisher model that distributes publishing behavior across your application graph instead of relying on a single top-level publisher.

Rather than selecting a target environment (like Docker or Azure) by calling `AddDockerComposePublisher()` or similar, Aspire now includes a **built-in publisher** that looks for a `PublishingCallbackAnnotation` on each resource. This annotation describes how that resource should be published—for example, as a Docker Compose service, Kubernetes manifest, or Azure Bicep module.

Tip

 This architectural shift lays the groundwork for **hybrid and heterogeneous deployments**, where different services within the same app can be deployed to different targets (cloud, edge, local).

Most apps only need one environment

In typical apps, you only need to add a **single compute environment**, like:

C#

```
builder.AddAzureContainerAppEnvironment("env");
```

In this case, Aspire applies the correct publishing behavior to all compute resources in your app model—no extra configuration needed.

Multiple environments require disambiguation

If you add **multiple compute environments**, Aspire needs to know which resource goes where. Compute environments apply their transformations to **all applicable compute resources** (projects, containers, executables). If more than one environment matches a given resource, Aspire throws an **ambiguous environment exception** at publish time.

You can resolve this by using `WithComputeEnvironment(...)`:

C#

```
var k8s = builder.AddKubernetesEnvironment("k8s-env");
var compose = builder.AddDockerComposeEnvironment("docker-env");

builder.AddProject<Projects.Api>("api")
    .WithComputeEnvironment(compose);

builder.AddProject<Projects.Frontend>("frontend")
    .WithComputeEnvironment(k8s);
```

This (contrived) example shows how you could explicitly map services to different compute targets—modeling, for example, a frontend in Kubernetes and a backend in Docker Compose.

Note

 Imagine a real-world case where your frontend is deployed to a CDN or GitHub Pages, and your backend runs in Azure Container Apps. This new model makes that future possible.

 All previous publisher registration APIs (like `AddDockerComposePublisher()`) have been removed in favor of this new model.

Supported compute environments

.NET Aspire 9.3 has preview support for the following environment resources:

- `AddDockerComposeEnvironment(...)`
- `AddKubernetesEnvironment(...)`
- `AddAzureContainerAppEnvironment(...)`
- `AddAzureAppServiceEnvironment(...)` — [see new App Service support →](#)

These represent deployment targets that can transform and emit infrastructure-specific artifacts from your app model.

Docker Compose enhancements

.NET Aspire 9.3 introduces powerful new capabilities for customizing Docker Compose output using strongly typed, C#-based configuration. You can now declaratively configure both the **global Compose file** and individual **services** directly from the Aspire app model—making your deployment output easy to reason about, customize, and automate.

Customize the Compose file and service definitions

You can now programmatically configure the top-level Compose file and the behavior of each individual service using two new APIs:

- `ConfigureComposeFile(...)` — customize the `docker-compose.yml` metadata
- `PublishAsDockerComposeService(...)` — modify the generated service for any compute resource (like a container or project)

C#

```
builder.AddDockerComposeEnvironment("env")
    .WithProperties(env =>
    {
        env.BuildContainerImages = false; // skip image build step
    })
    .ConfigureComposeFile(file =>
    {
        file.Name = "aspire-ai-chat"; // sets the file name
    });

// Add a container to the app
builder.AddContainer("service", "nginx")
    .WithEnvironment("ORIGINAL_ENV", "value")
    .PublishAsDockerComposeService((resource, service) =>
    {
        service.Labels["custom-label"] = "test-value";
        service.AddEnvironmentalVariable("CUSTOM_ENV", "custom-value");
        service.Restart = "always";
    });
```

These APIs give you a structured, strongly typed way to mutate the generated output—enabling richer CI automation, custom tooling, and environment-specific adjustments without editing YAML manually.

Map parameters and expressions into Docker Compose

.NET Aspire now supports **binding values from the app model**—like parameters and references—into the Docker Compose definition via environment variable placeholders.

This makes it easy to flow dynamic configuration (e.g., from the CI pipeline or secret store) directly into the final output.

C#

```
builder.AddDockerComposeEnvironment("docker-compose");

var containerNameParam = builder.AddParameter("param-1", "default-name",
publishValueAsDefault: true);

builder.AddContainer("service", "nginx")
    .WithEnvironment("ORIGINAL_ENV", "value")
    .PublishAsDockerComposeService((resource, service) =>
    {
        service.ContainerName =
        containerNameParam.AsEnvironmentPlaceholder(resource);
    });
```

The key API here is `.AsEnvironmentPlaceholder(...)`, which tells Aspire to emit a Compose variable like `${PARAM_1}` and register the mapping so the `.env` file is updated accordingly.

Tip

 This tightly couples your infrastructure parameters with the Docker Compose model—without hardcoding values—unlocking composability across environments.

These enhancements make Docker Compose a **fully programmable publishing target**, ideal for local development, container-based CI workflows, and teams that need structured control without brittle YAML overlays.

Kubernetes manifest customization

.NET Aspire 9.3 adds support for **programmatically customizing generated Kubernetes manifests** as part of the publish process. This gives you fine-grained control over the YAML artifacts Aspire emits—without writing raw manifest overlays or patches.

Like Docker Compose, Aspire now supports both **global environment-level settings** and **per-resource customization**.

Configure global and per-resource settings

You can use the following APIs to configure Kubernetes output in C#:

- `WithProperties(...)` on the compute environment to set global behaviors
- `PublishAsKubernetesService(...)` on compute resources to modify their specific Kubernetes resources

C#

```
builder.AddKubernetesEnvironment("env")
    .WithProperties(env =>
    {
        env.DefaultImagePullPolicy = "Always"; // e.g., Always, IfNotPresent
    });

builder.AddContainer("service", "nginx")
    .WithEnvironment("ORIGINAL_ENV", "value")
    .PublishAsKubernetesService(resource =>
    {
        // Add custom deployment-level settings
        resource.Deployment!.Spec.RevisionHistoryLimit = 5;
    });
```

This gives you fully typed access to the Kubernetes object model, enabling powerful modifications like:

- Overriding container image pull policies
- Customizing replica counts or deployment strategies
- Injecting labels or annotations into Services, Deployments, or ConfigMaps

Important

 Aspire emits standard Kubernetes manifests under the hood—you can still use `kubectl`, `helm`, or GitOps workflows to deploy them, but now you can shape them directly from your app definition.

Aspire CLI enhancements

 The Aspire CLI is **still in preview** and under active development. Expect more features and polish in future releases.

 To install:

Bash

```
dotnet tool install --global aspire.cli --prerelease
```

Note

 **The Aspire 9.3 CLI is not compatible with Aspire 9.2 projects.** You must upgrade your project to Aspire 9.3+ in order to use the latest CLI features.

Smarter app host discovery

The CLI now **walks upward** from your current directory, **recursively searching each level** for the app host project. Once located, it caches the result in a `.aspire` folder to speed up future commands.

You can now run commands like `aspire run`, `aspire add`, or `aspire publish` from **any directory within your solution**, and the CLI will resolve the app host automatically.

For example:

Bash

```
cd src/frontend
aspire run
```

Health-aware dashboard launch

The CLI now **waits for the dashboard to become responsive** before printing its URL to the terminal. This ensures the link works immediately when opened—no more blank pages or retry loops.

These updates make the Aspire CLI more reliable, script-friendly, and aligned with how developers move across folders and projects during daily development.

Azure goodies

Azure App Service (Preview support)

.NET Aspire 9.3 introduces **preview support for deploying .NET projects to Azure App Service**—one of the most requested features from developers using Aspire with existing Azure environments.

This integration lets you deploy your project as a **containerized Linux Web App**, modeled directly in your Aspire app host using a new `AddAzureAppServiceEnvironment(...)` API.

Current limitations (Preview)

This first release is scoped to the most common use cases:

- Supports **.NET projects only** (via `AddProject(...)`)
- Each project must expose a **single public HTTP endpoint**
- **Projects are published as containers** to Azure Container Registry
- **Containers within the app host are not supported**
- **Existing App Service Plans are not supported**
- The Aspire **dashboard is not hosted** in App Service yet

Important

 Hosted dashboard support is coming soon—we're actively developing this. Feedback is welcome!

Example: Deploy to Azure App Service

C#

```
builder.AddAzureAppServiceEnvironment("env");

builder.AddProject<Projects.Api>("api")
    .WithExternalHttpEndpoints()
    .PublishAsAzureAppServiceWebsite((infra, site) =>
    {
        site.SiteConfig.IsWebSocketsEnabled = true;
    });
```

In this example:

- Aspire provisions an App Service Plan and a Web App
- Your project is built as a container and published to **Azure Container Registry**
- The container is deployed to App Service with the configuration you provide

 Use `PublishAsAzureAppServiceWebsite(...)` to customize settings like site config, authentication, or SKU.

 This feature is in **preview**—we're looking for your feedback as we expand support!

Use an existing Azure Container Registry (ACR)

.NET Aspire 9.3 adds support for modeling an existing **Azure Container Registry (ACR)** using the new `AddAzureContainerRegistry(...)` API. This enables you to **push images to an ACR you already manage**—without Aspire provisioning a new one.

This is ideal for teams that:

- Share a centralized registry across environments
- Integrate with existing CI/CD pipelines and promotion workflows
- Require fine-grained control over image publishing

Example: associate ACR with an Azure Container Apps environment

C#

```
var acr = builder.AddAzureContainerRegistry("my-acr");

builder.AddAzureContainerAppEnvironment("env")
    .WithAzureContainerRegistry(acr);

builder.AddProject<Projects.Api>("api")
    .WithExternalHttpEndpoints();
```

In this example:

- The ACR is modeled in Aspire and used by the container apps environment
- Aspire publishes the built image to `my-acr` and configures Azure Container Apps to pull from it

ACR works with multiple compute environments

You can associate an `AzureContainerRegistryResource` with:

- `AddAzureContainerAppEnvironment(...)`
- `AddAzureAppServiceEnvironment(...)`

This gives you consistent control over where images are published, even across different compute targets.

💡 Use `.RunAsExisting()` or `.PublishAsExisting()` on the ACR resource to reference an existing registry without provisioning one.

Resource Deep Linking for Blob Containers

.NET Aspire 9.3 expands **resource deep linking** to include **Azure Blob Storage** containers, building on the model already used for Cosmos DB, Event Hubs, Service Bus, and Azure OpenAI.

You can now model individual blob containers directly in your app host, then inject scoped `BlobContainerClient` instances into your services—making it easy to read or write blobs without manually configuring connection strings or access.

AppHost:

```
C#

var builder = DistributedApplication.CreateBuilder(args);

// Add Azure Storage Emulator
var storage = builder.AddAzureStorage("storage").RunAsEmulator();

// Add a blob group and a container
var blobs = storage.AddBlobs("blobs");
var container = blobs.AddBlobContainer("images", blobContainerName: "image-uploads");

// Add the API project and reference the container
builder.AddProject<Projects.my94app_ApiService>("api")
    .WithExternalHttpEndpoints()
    .WithReference(container);

builder.Build().Run();
```

In the API project:

```
C#

using Azure.Storage.Blobs;

var builder = WebApplication.CreateBuilder(args);

// Register the blob container client
builder.AddAzureBlobContainerClient("images");
```

```

var app = builder.Build();

// Minimal POST endpoint for image upload
app.MapPost("/upload", async (
    IFormFile file,
    BlobContainerClient container) =>
{
    await container.CreateIfNotExistsAsync();

    var blob = container.GetBlobClient(file.FileName);
    using var stream = file.OpenReadStream();
    await blob.UploadAsync(stream, overwrite: true);

    return Results.Ok(new { Url = blob.Uri });
});

app.Run();

```

This pattern provides clean separation of concerns, secure container scoping, and minimal ceremony—ideal for microservices that interact with specific blob containers.

Expanded Azure Key Vault client integrations

.NET Aspire 9.3 expands Azure Key Vault support with new client integration APIs for **keys** and **certificates**, allowing you to inject typed Azure SDK clients directly into your services:

- `AddAzureKeyVaultKeyClient(...)`
- `AddAzureKeyVaultCertificateClient(...)`
- `AddKeyedAzureKeyVaultKeyClient(...)`
- `AddKeyedAzureKeyVaultCertificateClient(...)`

These APIs complement the existing `AddAzureKeyVaultClient(...)` and provide easy access to `KeyClient` and `CertificateClient` from the Azure SDK for .NET.

C#

```

var builder = WebApplication.CreateBuilder(args);

// Register default clients
builder.AddAzureKeyVaultKeyClient("kv");
builder.AddAzureKeyVaultCertificateClient("kv");

// Register named (keyed) clients
builder.AddKeyedAzureKeyVaultCertificateClient("kv", "signing-cert");

```

The **keyed overloads** allow you to register multiple clients scoped to the same Key Vault resource—useful when accessing multiple certificates or keys by purpose.



This feature was contributed by [@james-gould](#). Thank you!



Use Key Vault secrets in environment variables

.NET Aspire 9.3 adds support for wiring **Key Vault secrets directly into environment variables** using a new overload of `WithEnvironment(...)` that accepts an `IAzureKeyVaultSecretReference`.

This makes it easy to securely reference secrets from a modeled Key Vault without hardcoding secret values—and ensures those references flow correctly into deployment outputs like Azure Bicep.

C#

```
var kv = builder.AddAzureKeyVault("myKeyVault");

var secretRef = kv.Resource.GetSecret("mySecret");

builder.AddContainer("myContainer", "nginx")
    .WithEnvironment("MY_SECRET", secretRef);
```



Reference secrets from existing Key Vaults

You can also use this with **existing Azure Key Vaults** by marking the resource with `AsExisting(...)`, `RunAsExisting(...)`, or `PublishAsExisting(...)`. This lets you consume secrets from **already-provisioned vaults**—perfect for shared environments or team-managed infrastructure.

C#

```
var keyVaultNameParam = builder.AddParameter("key-vault-name");
var keyVaultResourceGroupParam = builder.AddParameter("key-vault-rg");

var existingVault = builder.AddAzureKeyVault("sharedVault")
    .AsExisting(keyVaultNameParam,
keyVaultResourceGroupParam);

var apiKey = existingVault.Resource.GetSecret("stripe-api-key");

builder.AddContainer("billing", "mycompany/billing")
    .WithEnvironment("STRIPE_API_KEY", apiKey);
```

This pattern ensures Aspire:

- Doesn't attempt to re-provision the Key Vault
- Emits references to the correct existing resources in publish mode

- Still enables secret injection and secure scoping via environment variables

📖 See also: [Use existing Azure resources](#).

Azure AI Inference client integration (Preview)

.NET Aspire 9.3 adds **client-only support for Azure-hosted Chat Completions endpoints** using the [Azure.AI.Inference](#) library and the [Microsoft.Extensions.AI](#) abstractions.

This integration simplifies calling Azure OpenAI or Azure AI Inference services from your application—whether you prefer working directly with the SDK or using abstraction-friendly interfaces.

Use `ChatCompletionsClient` with the Azure SDK

```
C#

builder.AddAzureChatCompletionsClient("connectionName");

app.MapPost("/chat-raw", (
    ChatCompletionsClient client,
    ChatRequest message) =>
{
    // Use the client
});
```

Use `IChatClient` via `Microsoft.Extensions.AI`

```
C#

builder.AddAzureChatCompletionsClient("inference")
    .AddChatClient();
```

Once registered, you can inject [IChatClient](#) using standard dependency injection:

```
C#

app.MapPost("/chat", async (
    IChatClient chatClient,
    ChatRequest message) =>
{
    var result = await chatClient.GetResponseAsync(message.Input);
    return result;
});
```

This setup integrates seamlessly with frameworks like [Semantic Kernel](#), and works well in modular or pluggable AI systems.

 Learn more about [Microsoft.Extensions.AI](#) and [ChatCompletionsClient](#).

Azure App Configuration client integration

.NET Aspire 9.3 adds support for **Azure App Configuration** via a new client integration,  [Aspire.Microsoft.Extensions.Configuration.AzureAppConfiguration](#) NuGet package.

This makes it easy to connect to centralized configuration using the official Azure SDK and the `Microsoft.Extensions.Configuration.AzureAppConfiguration` provider—no manual setup required.

C#

```
builder.AddAzureAppConfiguration("appconfig");
```

Once registered, Aspire automatically wires Azure App Configuration into your application's configuration pipeline.

Example: bind Azure App Configuration to app settings

C#

```
var builder = WebApplication.CreateBuilder(args);

builder.AddAzureAppConfiguration("appconfig");

var app = builder.Build();

app.MapGet("/feature", (IConfiguration config) =>
{
    var isEnabled = config.GetValue<bool>("FeatureFlag:Enabled");
    return Results.Ok(new { Enabled = isEnabled });
});

app.Run();
```

This enables:

- Dynamic feature flag evaluation
- Centralized configuration management across environments
- Secure integration into the Aspire hosting model

 Like all Azure integrations in Aspire, the App Configuration client defaults to using **Managed Identity** for secure access—no connection strings required.

 NuGet package: [Aspire.Microsoft.Extensions.Configuration.AzureAppConfiguration](https://www.nuget.org/packages/Aspire.Microsoft.Extensions.Configuration.AzureAppConfiguration)  
Learn more about [Azure App Configuration](#)

Secure multi-app access to Azure SQL (Breaking change)

In .NET Aspire 9.2, using **multiple projects with the same Azure SQL Server** inside an **Azure Container Apps environment** could silently break your app's identity model.

Each app was assigned its own **managed identity**, but Aspire granted **admin access** to the last app deployed—overwriting access for any previously deployed apps. This led to confusing failures where only one app could talk to the database at a time.

New behavior in 9.3

.NET Aspire 9.3 fixes this by:

1. Assigning **one identity** as the **SQL Server administrator**
2. Emitting a **SQL script** that:
 - Creates a **user** for each additional managed identity
 - Assigns each user the **db_owner** role on the target database

This ensures every app that references the database gets **full access** without conflicting with other apps.

Why this matters

- Supports **multiple apps accessing the same SQL Server** securely
- Preserves **least-privilege separation** across app identities
- Avoids the brittle “last one wins” admin behavior from earlier releases
- Enables richer deployment scenarios in cloud-native environments like Azure Container Apps

Breaking change

If your deployment relied on Aspire setting the managed identity as the SQL Server **admin**, you'll need to review your access model. Apps now receive **explicit role-based access** (**db_owner**) instead of broad admin rights.

 Related: [dotnet/aspire#8381](#) and [dotnet/aspire#8389](#)

Default Azure SQL SKU now uses the Free Offer (Breaking change)

.NET Aspire 9.3 changes the default SKU used when provisioning **Azure SQL databases** to the **GP_S_Gen5_2** (General Purpose Serverless) tier with the **Free Offer**. This helps reduce unexpected costs during development and experimentation.

Previously, Aspire defaulted to the **General Purpose (GP)** tier *without* the Free Offer, which could incur charges even for small or test apps.

What's new

When you provision a SQL database like this:

```
C#  
  
var sql = builder.AddAzureSqlServer("sqlserver");  
  
sql.AddDatabase("appdb");
```

Aspire now automatically uses the **Free Offer** for `appdb`, which will deploy a **GP_S_Gen5_2** (General Purpose Serverless), unless you override it.

How to restore the previous behavior

If your app requires the performance or features of the General Purpose paid tier, you can opt out of the new default using:

```
C#  
  
sql.AddDatabase("appdb")  
    .WithDefaultAzureSku(); // Uses the previous (General Purpose) default
```

If you want to specify what SKU to use, you the `ConfigureInfrastructure` method as explained here: [Setting a specific SKU](#).

Breaking change

This change affects cost, performance, and available features in new deployments. If your app depends on higher-tier capabilities, be sure to configure the SKU accordingly.

🔑 Use `.WithDefaultAzureSku()` on the `database` to revert to the old behavior

AZD: Major Improvements to CI/CD for Aspire Apps

We've dramatically improved how `azd` configures CI/CD pipelines for Aspire-based applications. These updates directly address one of the most frustrating pain points reported by the community: managing environment parameters and secrets securely and predictably across environments.

Aspire apps are increasingly parameter-driven — using infrastructure-defined settings like connection strings, runtime versions, API keys, and feature flags. Getting those values safely and consistently into CI pipelines like GitHub Actions has historically been difficult. This release fixes that.

Smarter Parameter Handling — No More `AZD_INITIAL_ENVIRONMENT_CONFIG`

Previously, Aspire apps that required infrastructure parameters relied on a hidden environment variable called `AZD_INITIAL_ENVIRONMENT_CONFIG`. This variable was a large JSON blob containing all local environment configuration. It had to be passed manually into CI pipelines, was difficult to inspect, and introduced friction when sharing or updating environments.

Now: `azd` extracts Aspire parameters directly from your infrastructure definitions and exposes them as named environment variables or secrets in your pipeline — **securely and explicitly**.

For example:

Bicep

```
param openaiKey string
param dbPassword string
```

become:

YAML

```
AZURE_OPENAI_KEY: ${ secrets.AZURE_OPENAI_KEY }
AZURE_DB_PASSWORD: ${ secrets.AZURE_DB_PASSWORD }
```

This means no more bundling, no more fragile config hacks, and no more guessing how your environment is configured in CI.



Consistent, Predictable Parameter Naming

Aspire parameters are mapped to environment variable names using a clear rule:

1. Convert `camelCase` to `SNAKE_CASE`
2. Replace dashes (`-`) with underscores (`_`)
3. Uppercase everything
4. Prefix with `AZURE_`

[Expand table](#)

Parameter name	Env var name
<code>openaiKey</code>	<code>AZURE_OPENAI_KEY</code>
<code>dbPassword</code>	<code>AZURE_DB_PASSWORD</code>
<code>storage-account-name</code>	<code>AZURE_STORAGE_ACCOUNT_NAME</code>

This naming consistency means Aspire deployment targets like Azure Container Apps can resolve configuration without custom mappings — locally or in the cloud.



Aspire Parameters Automatically Exported to CI

Aspire apps often define required parameters in Bicep or infrastructure modules — including things like API keys, credentials, or runtime configuration. These are now automatically exported to your pipeline configuration using the naming rules above.

You no longer need to:

- Manually configure these in `.azure/env-name/config.json`
- Inject them into CI via complex JSON blobs
- Worry about missing or mismatched configuration between local and cloud

Secure parameters (like `openaiKey` or `dbPassword`) are automatically treated as CI secrets, while others are injected as variables — all handled by `azd`.



Interactive Secret Management in GitHub Actions

When you run `azd pipeline config`, `azd` will now detect and prompt you if a secret already exists in your GitHub repo or if a secret is no longer used:

Existing Secret Prompt:

The secret `AZURE_OPENAI_KEY` already exists. What would you like to do?

- [1] Keep it
- [2] Keep ALL existing secrets
- [3] Overwrite it
- [4] Overwrite ALL secrets

Unused Secret Prompt:

The secret `AZURE_OLD_SECRET` is no longer used. What would you like to do?

- [1] Keep it
- [2] Keep ALL unused secrets
- [3] Delete it
- [4] Delete ALL unused secrets

This ensures:

- You're never surprised by secret overwrites
- You can keep your repo clean of stale configuration
- CI reflects your actual infrastructure setup



End-to-End, Repeatable Aspire Deployment

With these changes, the local-to-cloud workflow for Aspire apps is now consistent and automated:

1. You define infrastructure parameters as part of your Aspire app.
2. `azd` captures them during provisioning.
3. `azd pipeline config` maps them into your GitHub Actions or Azure DevOps pipeline.
4. Your pipeline runs securely with all the same inputs as your local environment — no manual steps required.

No more `AZD_INITIAL_ENVIRONMENT_CONFIG`. No more brittle overrides. Just clear, secure, parameterized deployment.

These changes unlock a smoother, safer CI/CD experience for Aspire projects — reducing manual configuration, improving security, and aligning your local development setup with your production pipeline.



Breaking changes

With every release, we strive to make .NET Aspire better. However, some changes may break existing functionality. The following breaking changes are introduced in .NET Aspire 9.3:

- [Breaking changes in .NET Aspire 9.3](#)

What's new in .NET Aspire 9.2

Article • 04/10/2025

 .NET Aspire 9.2 is the next minor version release of .NET Aspire; it supports:

- .NET 8.0 Long Term Support (LTS)
- .NET 9.0 Standard Term Support (STS)

If you have feedback, questions, or want to contribute to .NET Aspire, collaborate with us on [GitHub](#) or join us on [Discord](#) to chat with team members.

It's important to note that .NET Aspire releases out-of-band from .NET releases. While major versions of .NET Aspire align with .NET major versions, minor versions are released more frequently. For more information on .NET and .NET Aspire version support, see:

- [.NET support policy](#): Definitions for LTS and STS.
- [.NET Aspire support policy](#): Important unique product life cycle details.

Upgrade to .NET Aspire 9.2

Important

If you are using `azd` to deploy Azure PostgreSQL or Azure SQL Server, you now have to configure Azure Managed Identities. For more information, see [Improved Managed Identity defaults](#).

Moving between minor releases of .NET Aspire is simple:

1. In your app host project file (that is, *MyApp.AppHost.csproj*), update the  [Aspire.AppHost.Sdk](#) NuGet package to version `9.2.0`:

diff

```
<Project Sdk="Microsoft.NET.Sdk">

  <Sdk Name="Aspire.AppHost.Sdk" Version="9.2.0" />

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net9.0</TargetFramework>
    - <IsAspireHost>true</IsAspireHost>
    <!-- Omitted for brevity -->
  </PropertyGroup>

  <ItemGroup>
```

```
<PackageReference Include="Aspire.Hosting.AppHost" Version="9.2.0" />
</ItemGroup>

<!-- Omitted for brevity -->
</Project>
```

Important

The `IsAspireHost` property is no longer required in the project file. For more information, see  [Project file changes](#).

For more information, see [.NET Aspire SDK](#).

2. Check for any NuGet package updates, either using the NuGet Package Manager in Visual Studio or the **Update NuGet Package** command in VS Code.
3. Update to the latest [.NET Aspire templates](#) by running the following .NET command line:

```
.NET CLI
```

```
dotnet new update
```

Important

The `dotnet new update` command updates all of your templates to the latest version.

If your app host project file doesn't have the `Aspire.AppHost.Sdk` reference, you might still be using .NET Aspire 8. To upgrade to 9.0, follow [the upgrade guide](#).

App host enhancements

The [app host](#) is the core of .NET Aspire, providing the local hosting environment for your distributed applications. In .NET Aspire 9.2, we've made several improvements to the app host:

Project file changes

The .NET Aspire app host project file no longer requires the `IsAspireHost` property. This property was moved to the `Aspire.AppHost.Sdk` SDK, therefore, you can remove it from your project file. For more information, see [dotnet/aspire issue #8144](#) .

Define custom resource URLs

Resources can now define custom URLs. This makes it easier to build custom experiences for your resources. For example, you can define a custom URL for a database resource that points to the database management console. This makes it easier to access the management console directly from the dashboard, you can even give it a friendly name.

C#

```
var builder = DistributedApplication.CreateBuilder(args);

var catalogDb = builder.AddPostgres("postgres")
    .WithDataVolume()
    .WithPgAdmin(resource =>
    {
        resource.WithUrlForEndpoint("http", u => u.DisplayText
    = "PG Admin");
    })
    .AddDatabase("catalogdb");
```

The preceding code sets the display text for the `PG Admin` URL to `PG Admin`. This makes it easier to access the management console directly from the dashboard.

For more information, see [Define custom resource URLs](#).

Dashboard user experience improvements

.NET Aspire 9.2 adds new features to the [dashboard](#), making it a more powerful developer tool than ever. The following features were added to the dashboard in .NET Aspire 9.2:

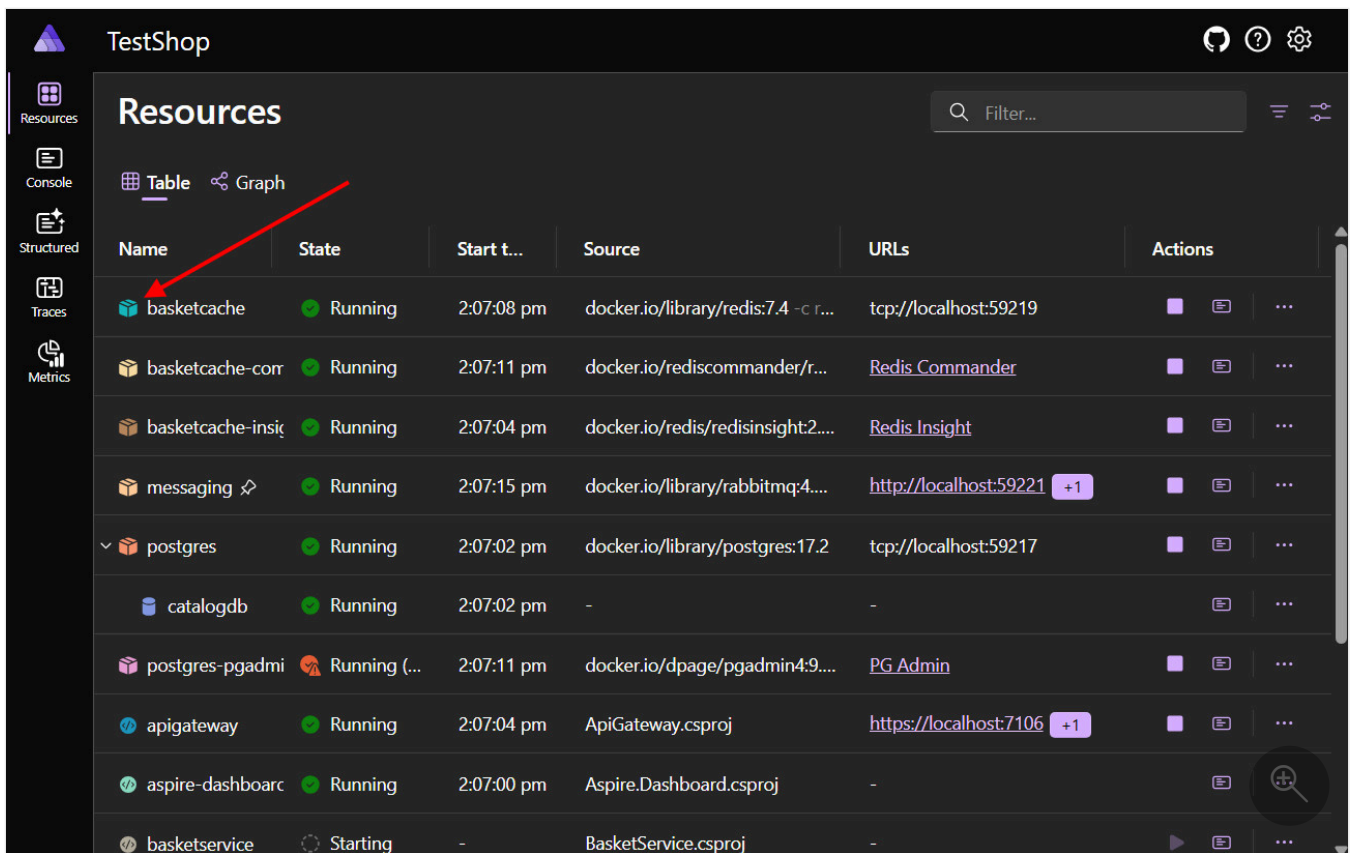
Resource graph

The resource graph is a new way to visualize the resources in your apps. It displays a graph of resources, linked by relationships. Click the 'Graph' tab on the Resources page to view the resource graph. See it in action on [James's BlueSky](#) .

For more information, see [.NET Aspire dashboard: Resources page](#).

Resource icons

We've added resource icons to the resources page. The icon color matches the resource's telemetry in structured logs and traces.



TestShop

Resources

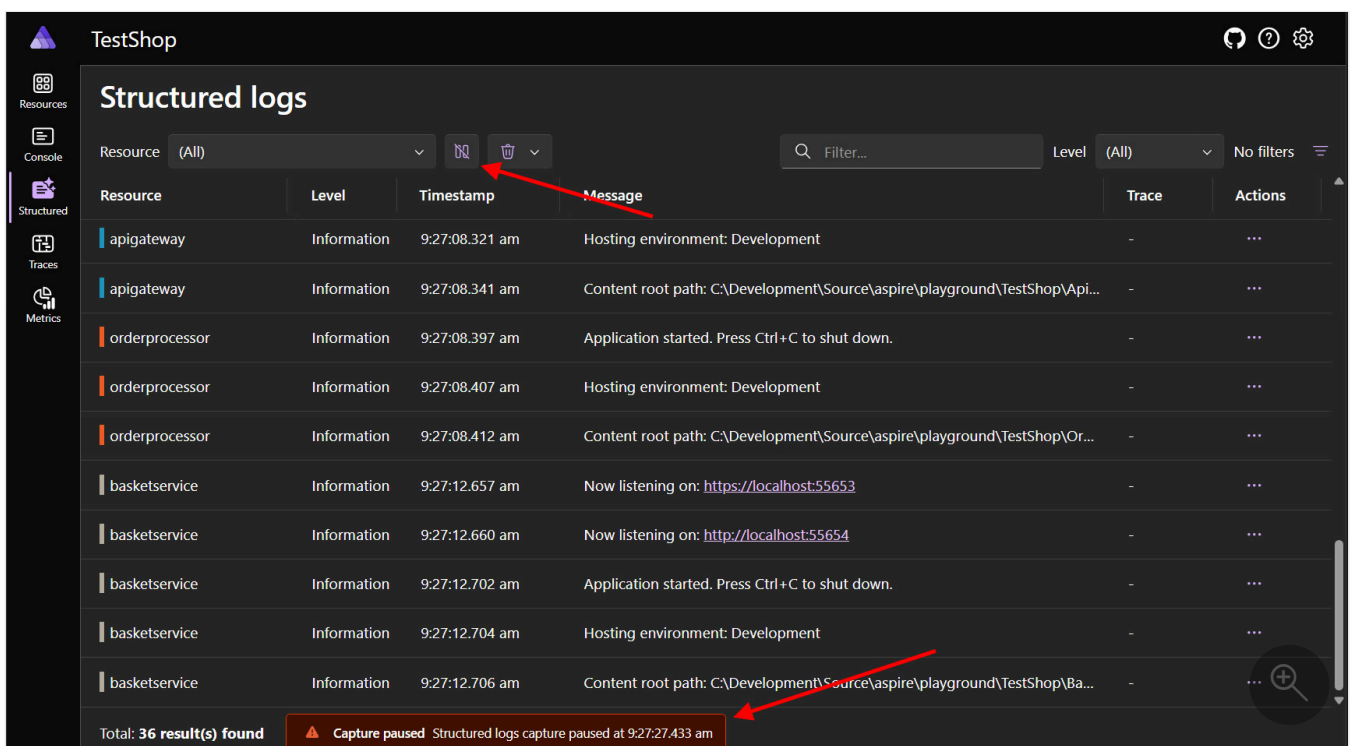
Table | Graph

Name	State	Start t...	Source	URLs	Actions
basketcache	Running	2:07:08 pm	docker.io/library/redis:7.4 -C r...	tcp://localhost:59219	Stop Logs ...
basketcache-corr	Running	2:07:11 pm	docker.io/rediscommander/r...	Redis Commander	Stop Logs ...
basketcache-insig	Running	2:07:04 pm	docker.io/redis/redisinsight2...	Redis Insight	Stop Logs ...
messaging	Running	2:07:15 pm	docker.io/library/rabbitmq:4...	http://localhost:59221 +1	Stop Logs ...
postgres	Running	2:07:02 pm	docker.io/library/postgres:17.2	tcp://localhost:59217	Stop Logs ...
catalogdb	Running	2:07:02 pm	-	-	Stop Logs ...
postgres-pgadmin	Running (...)	2:07:11 pm	docker.io/dpage/pgadmin4:9...	PG Admin	Stop Logs ...
apigateway	Running	2:07:04 pm	ApiGateway.csproj	https://localhost:7106 +1	Stop Logs ...
aspire-dashboard	Running	2:07:00 pm	Aspire.Dashboard.csproj	-	Stop Logs ...
basketservice	Starting	-	BasketService.csproj	-	Stop Logs ...

Pause and resume telemetry

New buttons were added to the **Console logs**, **Structured logs**, **Traces** and **Metrics** pages to pause collecting telemetry. Click the pause button again to resume collecting telemetry.

This feature allows you to pause telemetry in the dashboard while continuing to interact with your app.



TestShop

Structured logs

Resource (All) | Filter... | Level (All) | No filters

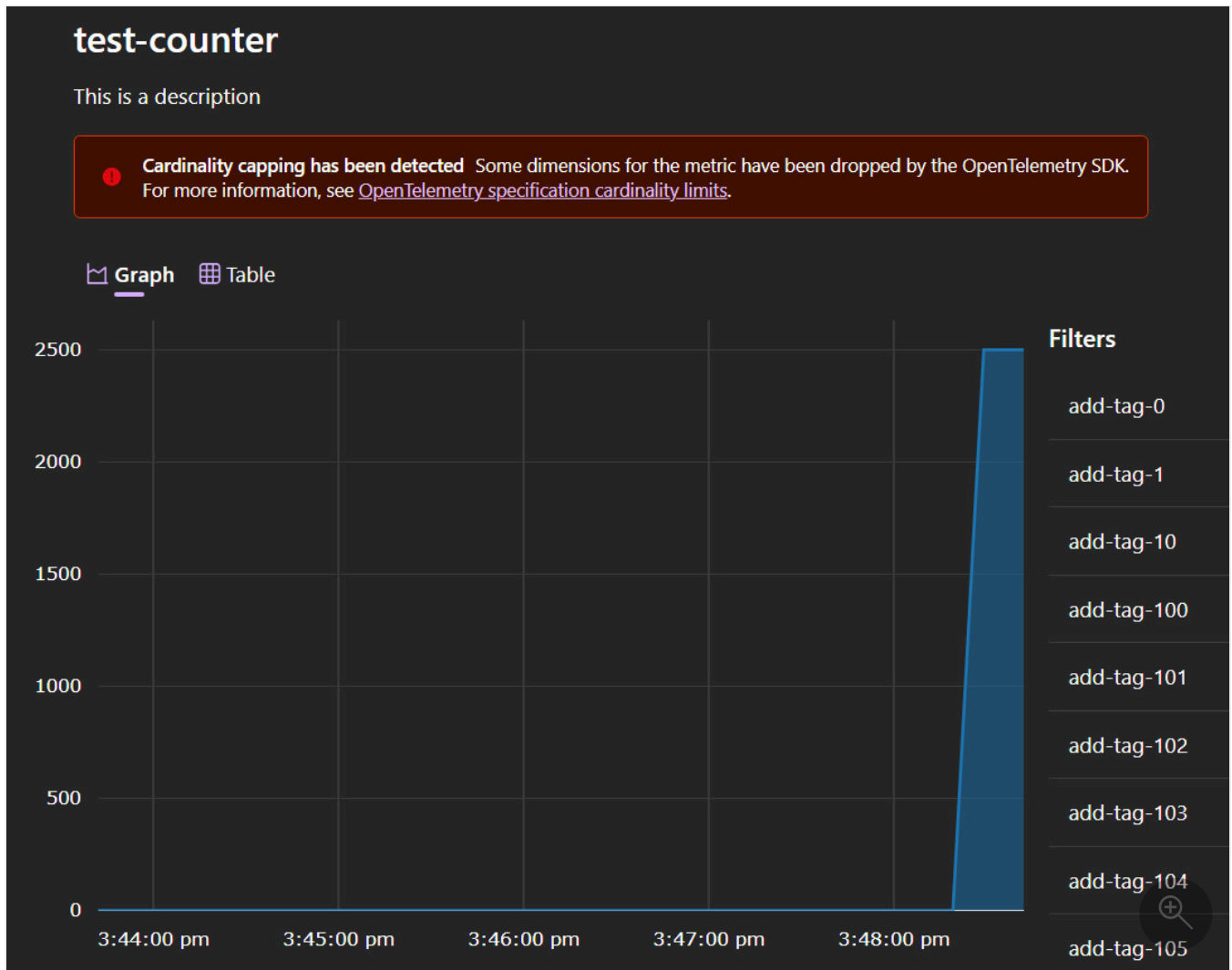
Resource	Level	Timestamp	Message	Trace	Actions
apigateway	Information	9:27:08.321 am	Hosting environment: Development	-	...
apigateway	Information	9:27:08.341 am	Content root path: C:\Development\Source\aspire\playground\TestShop\Api...	-	...
orderprocessor	Information	9:27:08.397 am	Application started. Press Ctrl+C to shut down.	-	...
orderprocessor	Information	9:27:08.407 am	Hosting environment: Development	-	...
orderprocessor	Information	9:27:08.412 am	Content root path: C:\Development\Source\aspire\playground\TestShop\Or...	-	...
basketservice	Information	9:27:12.657 am	Now listening on: https://localhost:55653	-	...
basketservice	Information	9:27:12.660 am	Now listening on: http://localhost:55654	-	...
basketservice	Information	9:27:12.702 am	Application started. Press Ctrl+C to shut down.	-	...
basketservice	Information	9:27:12.704 am	Hosting environment: Development	-	...
basketservice	Information	9:27:12.706 am	Content root path: C:\Development\Source\aspire\playground\TestShop\Ba...	-	...

Total: 36 result(s) found

Capture paused Structured logs capture paused at 9:27:27.433 am

📈🩹 Metrics health warning

The dashboard now warns you when a metric exceeds the configured cardinality limit. Once exceeded, the metric no longer provides accurate information.



🕒 UTC Console logs option

Console logs now supports UTC timestamps. The setting is accessible via the console logs options button.

The screenshot shows the TestShop console logs for a PostgreSQL resource. The logs display various messages, including database initialization and connection status. A red arrow highlights a specific log entry: "PostgreSQL Database directory appears to contain a database; Skipping init". A dropdown menu is open, showing options to "Download logs", "Hide timestamps", and "UTC timestamps".

Trace details search text box

We've added a search text box to trace details. Now you can quickly filter large traces to find the exact span you need. See it in action on [BluSky](#).

HTTP-based resource command functionality

[Custom resource commands](#) now support HTTP-based functionality with the addition of the `WithHttpCommand` API, enabling you to define endpoints for tasks like database migrations or resets. These commands can be run directly from the .NET Aspire dashboard.

Adds `WithHttpCommand()`, which lets you define a resource command that sends an HTTP request to your app during development. Useful for triggering endpoints like seed or reset from the dashboard.

C#

```
if (builder.Environment.IsDevelopment())
{
    var resetDbKey = Guid.NewGuid().ToString();

    catalogDbApp.WithEnvironment("DatabaseResetKey", resetDbKey)
        .WithHttpCommand("/reset-db", "Reset Database",
            commandOptions: new()
            {
```

```

        Description = "Reset the catalog database to its initial
state. This will delete and recreate the database.",
        ConfirmationMessage = "Are you sure you want to reset the
catalog database?",
        IconName = "DatabaseLightning",
        PrepareRequest = requestContext =>
        {
            requestContext.Request.Headers.Add("Authorization",
$"Key {resetDbKey}");
            return Task.CompletedTask;
        }
    });
}

```

For more information, see [Custom HTTP commands in .NET Aspire](#).

Connection string resource type

We've introduced a new `ConnectionStringResource` type that makes it easier to build dynamic connection strings without defining a separate resource type. This makes it easier to work with and build dynamic parameterized connection strings.

C#

```

var builder = DistributedApplication.CreateBuilder(args);

var apiKey = builder.AddParameter("apiKey");
var cs = builder.AddConnectionString("openai",
    ReferenceExpression.Create($"Endpoint=https://api.openai.com/v1;AccessKey=
{apiKey}"));

var api = builder.AddProject<Projects.Api>("api")
    .WithReference(cs);

```

Container resources can now specify an image pull policy

Container resources can now specify an `ImagePullPolicy` to control when the image is pulled. This is useful for resources that are updated frequently or that have a large image size. The following policies are supported:

- `Default`: Default behavior (which is the same as `Missing` in 9.2).
- `Always`: Always pull the image.
- `Missing`: Ensures the image is always pulled when the container starts.

C#


```
var builder = DistributedApplication.CreateBuilder(args);

var cache = builder.AddRedis("cache")
    .WithImageTag("latest")
    .WithImagePullPolicy(ImagePullPolicy.Always)
    .WithRedisInsight();
```

The `ImagePullPolicy` is set to `Always`, which means the image will always be pulled when the resource is created. This is useful for resources that are updated frequently.

New container files API

In .NET Aspire 9.2, we've added a new `WithContainerFiles` API, a way to create files and folders inside a container at runtime by defining them in code. Under the hood, it uses `docker cp` / `podman cp` to copy the files in. Supports setting contents, permissions, and ownership—no bind mounts or temp files needed.

Integrations updates

Integrations are a key part of .NET Aspire, allowing you to easily add and configure services in your app. In .NET Aspire 9.2, we've made several updates to integrations:

Redis/Valkey/Garnet: Password support enabled by default

The Redis, Valkey, and Garnet containers enable password authentication by default. This is part of our goal to be secure by default—protecting development environments with sensible defaults while still making them easy to configure. Passwords can be set explicitly or generated automatically if not provided.

Automatic database creation support

There's [plenty of feedback and confusion](#) around the `AddDatabase` API. The name implies that it adds a database, but it didn't actually create the database. In .NET Aspire 9.2, the `AddDatabase` API now creates a database for the following hosting integrations:

 Expand table

Hosting integration	API reference
 Aspire.Hosting.SqlServer	AddDatabase

Hosting integration	API reference
 Aspire.Hosting.PostgreSQL 	AddDatabase

The Azure SQL and Azure PostgreSQL hosting integrations also expose `AddDatabase` APIs which work with their respective `RunAsContainer` methods. For more information, see [Understand Azure integration APIs](#).

By default, .NET Aspire will create an empty database if it doesn't exist. You can also optionally provide a custom script to run during creation for advanced setup or seeding.

Example using Postgres:

C#

```
var builder = DistributedApplication.CreateBuilder(args);

var postgres = builder.AddPostgres("pg1");

postgres.AddDatabase("todoapp")
    .WithCreationScript($$"""
        CREATE DATABASE {{databaseName}}
        ENCODING = 'UTF8';
        """);
```

For more information and examples of using the `AddDatabase` API, see:

- [Add PostgreSQL resource with database scripts](#)
- [Add SQL Server resource with database scripts](#)

The following hosting integrations don't currently support database creation:

-  [Aspire.Hosting.MongoDb](#) 
-  [Aspire.Hosting.MySql](#) 
-  [Aspire.Hosting.Oracle](#) 

Azure integration updates

In .NET Aspire 9.2, we've made significant updates to Azure integrations, including:

Configure Azure Container Apps environments

.NET Aspire 9.2 introduces `AddAzureContainerAppEnvironment`, allowing you to define an Azure Container App environment directly in your app model. This adds an

`AzureContainerAppsEnvironmentResource` that lets you configure the environment and its supporting infrastructure (like container registries and volume file shares) using C# and the [Azure.Provisioning](#) APIs—without relying on `azd` for infrastructure generation.

Important

This uses a different resource naming scheme than `azd`. If you're upgrading an existing deployment, this may create duplicate resources. To avoid this, you can opt into `azd`'s naming convention:

```
C#  
  
var builder = DistributedApplication.CreateBuilder(args);  
  
builder.AddAzureContainerAppEnvironment("my-env")  
    .WithAzdResourceNaming();
```

For more information, see [Configure Azure Container Apps environments](#).

New Client integrations: Azure PostgreSQL (Npgsql & EF Core)

.NET Aspire 9.2 adds client integrations for working with **Azure Database for PostgreSQL**, supporting both local development and secure cloud deployment.

These integrations automatically use **Managed Identity (Entra ID)** in the cloud and during local development by default. They also support username/password, if configured in your AppHost. No application code changes are required to switch between authentication models.

-  [Aspire.Azure.Npgsql](#) 
-  [Aspire.Azure.Npgsql.EntityFrameworkCore.PostgreSQL](#) 

In AppHost:

```
C#  
  
var postgres = builder.AddAzurePostgresFlexibleServer("pg")  
    .AddDatabase("postgresdb");  
  
builder.AddProject<Projects.MyService>()  
    .WithReference(postgres);
```

In MyService:

C#

```
builder.AddAzureNpgsqlDbContext<MyDbContext>("postgresdb");
```

Resource Deep Linking for Cosmos DB, Event Hubs, Service Bus, and OpenAI

CosmosDB databases and containers, EventHub hubs, ServiceBus queues/topics, and Azure OpenAI deployments now support **resource deep linking**. This allows connection information to target specific child resources—like a particular **Cosmos DB container**, **Event Hubs**, or **OpenAI deployment**—rather than just the top-level account or namespace.

Hosting integrations preserve the full resource hierarchy in connection strings, and client integrations can resolve and inject clients scoped to those specific resources.

AppHost:

C#

```
var builder = DistributedApplication.CreateBuilder(args);

var cosmos = builder.AddAzureCosmosDB("cosmos")
    .RunAsPreviewEmulator(e => e.WithDataExplorer());

var db = cosmos.AddCosmosDatabase("appdb");
db.AddContainer("todos", partitionKey: "/userId");
db.AddContainer("users", partitionKey: "/id");

builder.AddProject<Projects.TodoApi>("api")
    .WithReference(db);
```

In the API project:

C#

```
var builder = WebApplication.CreateBuilder(args);

builder.AddAzureCosmosDatabase("appdb")
    .AddKeyedContainer("todos")
    .AddKeyedContainer("users");

app.MapPost("/todos", async ([FromKeyedServices("todos")] Container container,
    TodoItem todo) =>
{
    todo.Id = Guid.NewGuid().ToString();
    await container.CreateItemAsync(todo, new PartitionKey(todo.UserId));
}
```

```
return Results.Created($"{todos/{todo.Id}", todo);
});
```

This makes it easy and convenient to use the SDKs to interact with specific resources directly—without extra wiring or manual configuration. It's especially useful in apps that deal with multiple containers or Azure services.

Improved Managed Identity defaults

Starting in **.NET Aspire 9.2**, each Azure Container App now gets its **own dedicated managed identity** by default. This is a significant change from previous versions, where all apps shared a single, highly privileged identity.

This change strengthens Aspire's *secure by default* posture:

- Each app only gets access to the Azure resources it needs.
- It enforces the principle of least privilege.
- It provides better isolation between apps in multi-service environments.

By assigning identities individually, Aspire can now scope role assignments more precisely—improving security, auditability, and alignment with Azure best practices.

This is a **behavioral breaking change** and may impact apps using:

- **Azure SQL Server** - Azure SQL only supports one Azure AD admin. With multiple identities, only the *last deployed app* will be granted admin access by default. Other apps will need explicit users and role assignments.
- **Azure PostgreSQL** - The app that creates the database becomes the owner. Other apps (like those running migrations or performing data operations) will need explicit `GRANT` permissions to access the database correctly.

See the [breaking changes](#) page for more details.

This new identity model is an important step toward more secure and maintainable applications in Aspire. While it introduces some setup considerations, especially for database integrations, it lays the groundwork for better default security across the board.

Least-privilege role assignment functionality

.NET Aspire now supports APIs for modeling **least-privilege role assignments** when referencing Azure resources. This enables more secure defaults by allowing you to define exactly which roles each app needs for specific Azure resources.

C#

```
var builder = DistributedApplication.CreateBuilder(args);

var storage = builder.AddAzureStorage("storage")
    .RunAsEmulator(c =>
        c.WithLifetime(ContainerLifetime.Persistent));

var blobs = storage.AddBlobs("blobs");

builder.AddProject<Projects.AzureContainerApps_ApiService>("api")
    .WithExternalHttpEndpoints()
    .WithReference(blobs)
    .WithRoleAssignments(storage,
        StorageBuiltInRole.StorageBlobDataContributor);
```

In this example, the API project is granted **Storage Blob Data Contributor** only for the referenced storage account. This avoids over-provisioning permissions and helps enforce the principle of least privilege.

Each container app automatically gets its own **managed identity**, and Aspire now generates the necessary role assignment infrastructure for both default and per-reference roles. When targeting existing Azure resources, role assignments are scoped correctly using separate Bicep resources.

1 First-class Azure Key Vault Secret support

.NET Aspire now supports `IAzureKeyVaultSecretReference`, a new primitive for modeling secrets directly in the app model. This replaces `BicepSecretOutputReference` and gives finer grain control over Key Vault creation when using `AzureBicepResource`.

You can now:

- Add a shared Key Vault in C#
- Configure services that support keys (e.g., Redis, Cosmos DB) to store their secrets there
- Reference those secrets in your app as environment variables or via the Key Vault config provider

Use KeyVault directly in the "api" project:

C#

```
var builder = DistributedApplication.CreateBuilder(args);

var vault = builder.AddAzureKeyVault("kv");

var redis = builder.AddAzureRedis("redis")
```

```
        .WithAccessKeyAuthentication(vault);

builder.AddProject<Projects.Api>("api")
    .WithReference(vault);
```

Let the compute environment handle the secret management for you:

```
C#

var redis = builder.AddAzureRedis("redis")
    .WithAccessKeyAuthentication();

builder.AddProject<Projects.Api>("api")
    .WithReference(redis);
```

Previous behavior:

`azd` created and managed secret outputs using a key vault per resource, with no visibility in the app model. These Key Vault resources were handled implicitly and couldn't be customized in C#.

New behavior in 9.2:

Calling `WithKeyAccessAuthentication` or `WithPasswordAuthentication` now creates an actual `AzureKeyVaultResource` (or accepts a reference to one), and stores connection strings there. Secret names follow the pattern `connectionstrings--{resourcename}` to prevent naming conflicts with other vault entries.

Improved default permissions for Azure Key Vault references

When referencing a Key Vault, Aspire previously granted the broad **Key Vault Administrator** role by default. In 9.2, this has been changed to **Key Vault Secrets User**, which provides read-only access to secrets—suitable for most application scenarios.

This update continues the security-focused improvements in this release.

Deployment improvements

We're excited to announce several new deployment features in .NET Aspire 9.2, including:

Publishers (Preview)

Publishers are a new extensibility point in .NET Aspire that allow you to define how your distributed application gets transformed into deployable assets. Rather than relying on an [intermediate manifest format](#), publishers can now plug directly into the application model to generate Docker Compose files, Kubernetes manifests, Azure resources, or whatever else your environment needs.

When .NET Aspire launched, it introduced a deployment manifest format—a serialized snapshot of the application model. While useful it burdened deployment tools with interpreting the manifest and resource authors with ensuring accurate serialization. This approach also complicated schema evolution and target-specific behaviors.

Publishers simplify this process by working directly with the full application model in-process, enabling richer, more flexible, and maintainable publishing experiences.

The following NuGet packages expose preview publishers:

-  [Aspire.Hosting.Azure](#) 
-  [Aspire.Hosting.Docker \(Preview\)](#) 
-  [Aspire.Hosting.Kubernetes \(Preview\)](#) 

Important

The Docker and Kubernetes publishers were contributed by community contributor, [Dave Sekula](#) —a great example of the community stepping up to extend the model. 💜 Thank you, Dave!

To use a publisher, add the corresponding NuGet package to your app host project file and then call the `Add[Name]Publisher()` method in your app host builder.

C#

```
builder.AddDockerComposePublisher();
```

Tip

Publisher registration methods follow the `Add[Name]Publisher()` convention.

You can also build your own publisher by implementing the publishing APIs and calling your custom registration method. Some publishers are still in preview, and the APIs are subject to change. The goal is to provide a more flexible and extensible way to publish distributed applications, making it easier to adapt to different deployment environments and scenarios.



Aspire CLI (Preview)

.NET Aspire 9.2 introduces the new **aspire** CLI, a tool for creating, running, and publishing Aspire applications from the command line. It provides a rich, interactive experience tailored for Aspire users.

The CLI is available as a .NET tool and can be installed with:

Bash

```
dotnet tool install --global aspire.cli --prerelease
```

Example usage:

Bash

```
aspire new
aspire run
aspire add redis
aspire publish --publisher docker-compose
```

Available commands:

- **new** <template> – Create a new Aspire sample project
- **run** – Run an Aspire app host in development mode
- **add** <integration> – Add an integration to your project
- **publish** – Generate deployment artifacts from your app host



The CLI is **preview**. We're exploring how to make it a first-class experience for .NET Aspire users—your feedback is welcome!



Testing template updates

The xUnit testing project template now supports a version selector, allowing the user to select either:

- **v2**: The previous xUnit testing experience.
- **v3**: The new xUnit testing experience and template.
- **v3 with Microsoft Test Platform**: The next xUnit testing experience, template and uses the [Microsoft Testing Platform](#).

By default, to the `v3` experience. For more information, see:

- [What's new in xUnit v.3](#) 
- [Microsoft Testing Platform support in xUnit.net v3](#) 

 Note

Both `v3` versions are only supported with .NET Aspire 9.2 or later.

Breaking changes

With every release, we strive to make .NET Aspire better. However, some changes may break existing functionality. The following breaking changes are introduced in .NET Aspire 9.2:

- [Breaking changes in .NET Aspire 9.2](#)

What's new in .NET Aspire 9.1

Article • 04/15/2025

 .NET Aspire 9.1 is the next minor version release of .NET Aspire; it supports *both*:

- .NET 8.0 Long Term Support (LTS) *or*
- .NET 9.0 Standard Term Support (STS).

Note

You're able to use .NET Aspire 9.1 with either .NET 8 or .NET 9!

As always, we focused on highly requested features and pain points from the community. Our theme for 9.1 was "polish, polish, polish"—so you see quality of life fixes throughout the whole platform. Some highlights from this release are resource relationships in the dashboard, support for working in GitHub Codespaces, and publishing resources as a Dockerfile.

If you have feedback, questions, or want to contribute to .NET Aspire, collaborate with us on  [GitHub](#) or join us on  [Discord](#) to chat with team members.

Whether you're new to .NET Aspire or have been with us since the preview, it's important to note that .NET Aspire releases out-of-band from .NET releases. While major versions of .NET Aspire align with .NET major versions, minor versions are released more frequently. For more details on .NET and .NET Aspire version support, see:

- [.NET support policy](#): Definitions for LTS and STS.
- [.NET Aspire support policy](#): Important unique product life cycle details.

Upgrade to .NET Aspire 9.1

Moving between minor releases of .NET Aspire is simple:

1. In your app host project file (that is, *MyApp.AppHost.csproj*), update the  [Aspire.AppHost.Sdk](#) NuGet package to version **9.1.0**:

XML

```
<Project Sdk="Microsoft.NET.Sdk">

  <Sdk Name="Aspire.AppHost.Sdk" Version="9.1.0" />

  <!-- Omitted for brevity -->
```

</Project>

For more information, see [.NET Aspire SDK](#).

2. Check for any NuGet package updates, either using the NuGet Package Manager in Visual Studio or the **Update NuGet Package** command in VS Code.
3. Update to the latest [.NET Aspire templates](#) by running the following .NET command line:

```
.NET CLI
```

```
dotnet new update
```

⚠ Note

The `dotnet new update` command updates all of your templates to the latest version.

If your app host project file doesn't have the `Aspire.AppHost.Sdk` reference, you might still be using .NET Aspire 8. To upgrade to 9.0, you can follow [the documentation from last release](#).

Improved onboarding experience

The onboarding experience for .NET Aspire is improved with 9.1. The team worked on creating a GitHub Codespaces template that installs all the necessary dependencies for .NET Aspire, making it easier to get started, including the templates and the ASP.NET Core developer certificate. Additionally, there's support for Dev Containers. For more information, see:

- [.NET Aspire and GitHub Codespaces](#)
- [.NET Aspire and Visual Studio Code Dev Containers](#)

Dashboard UX and customization

With every release of .NET Aspire, the [dashboard](#) gets more powerful and customizable, this release is no exception. The following features were added to the dashboard in .NET Aspire 9.1:

Resource relationships

The dashboard now supports "parent" and "child" resource relationships. For instance, when you create a Postgres instance with multiple databases, these databases are nested under the same instance on the **Resource** page.

The screenshot shows a web browser window with the TestShop dashboard. The dashboard has a dark theme and a header with the TestShop logo and a menu icon. The main content area is titled 'Resources' and contains a table with the following columns: Name, State, Endpoints, and Actions. The table lists several resources, all of which are in a 'Running' state. A red arrow points to the 'catalogdb' resource.

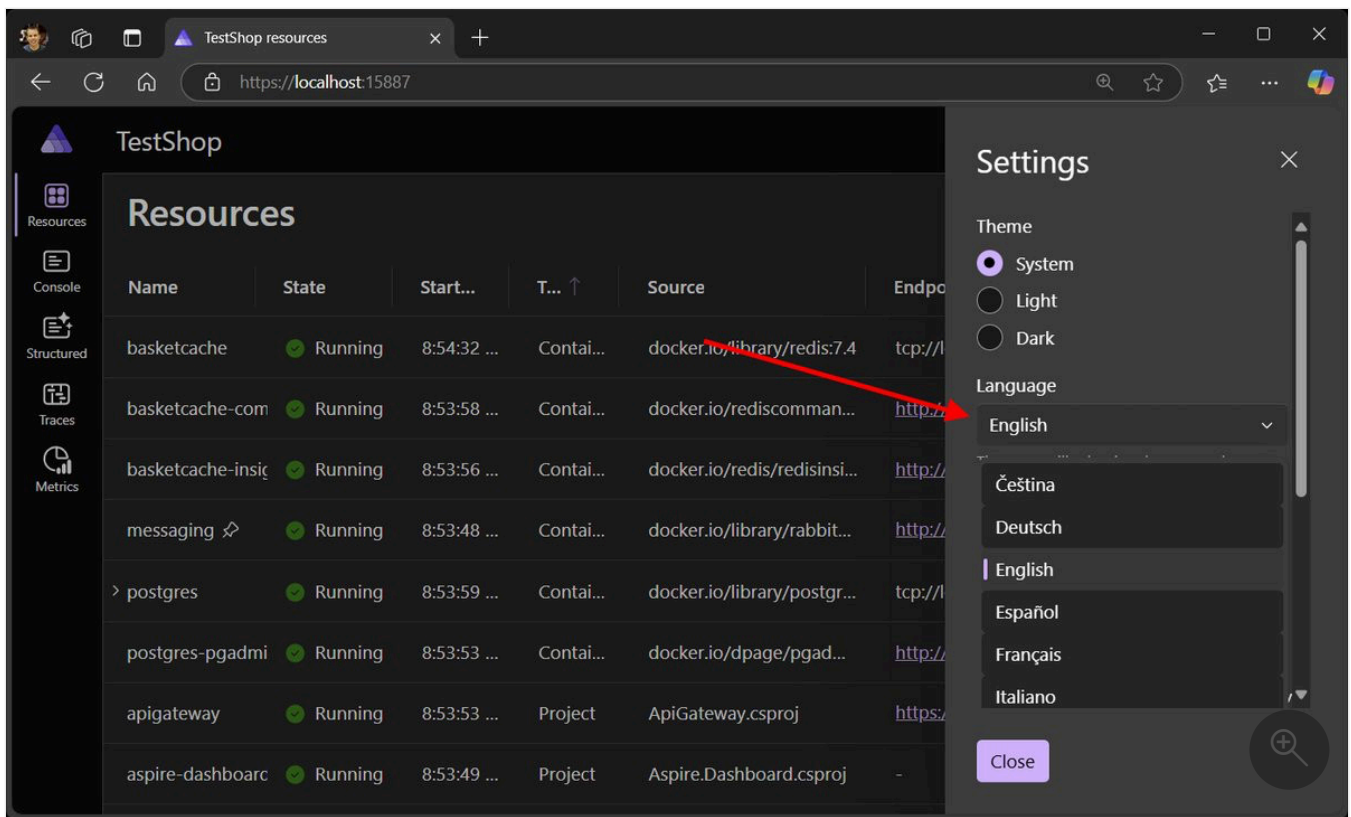
Name	State	Endpoints	Actions
messaging	✓ Running	http://localhost:59608 +1	...
▼ postgres	✓ Running	tcp://localhost:59604	...
catalogdb	✓ Running	-	...
postgres-pgadmin	✓ Running	http://localhost:59605	...
apigateway	✓ Running	https://localhost:7106 +1	...
aspire-dashboard	✓ Running	-	...
basketervice	✓ Running	https://localhost:5305 +1	...

View filters

For more information, see [Explore the .NET Aspire dashboard](#).

Localization overrides

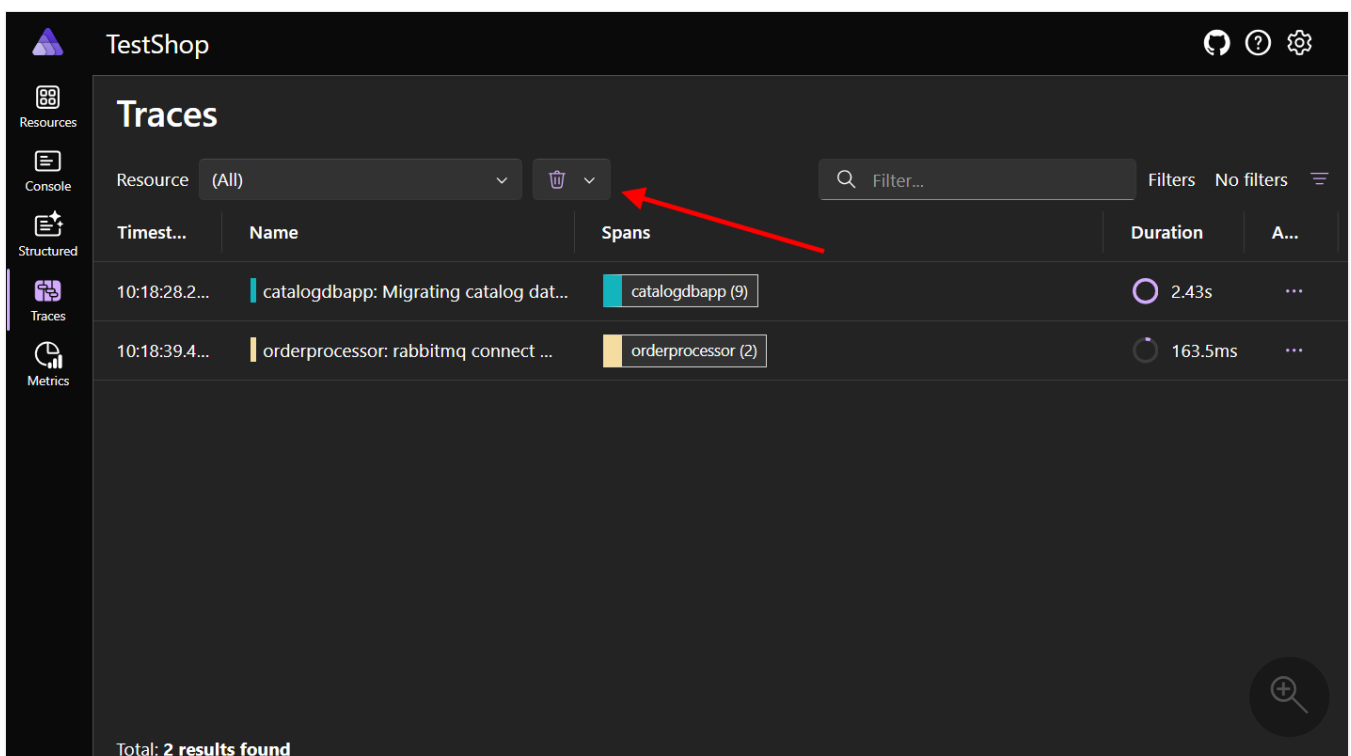
The dashboard defaults to the language set in your browser. This release introduces the ability to override this setting and change the dashboard language independently from the browser language. Consider the following screen capture that demonstrates the addition of the language dropdown in the dashboard:



Clear logs and telemetry from the dashboard

New buttons were added to the **Console logs**, **Structured logs**, **Traces** and **Metrics** pages to clear data. There's also a "Remove all" button in the settings popup to remove everything with one action.

Now you use this feature to reset the dashboard to a blank slate, test your app, view only the relevant logs and telemetry, and repeat.



We ❤️ love the developer community and thrive on its feedback, collaboration, and contributions. This feature is a community contribution from [@Daluur](#). Join us in celebrating their contribution by using the feature!

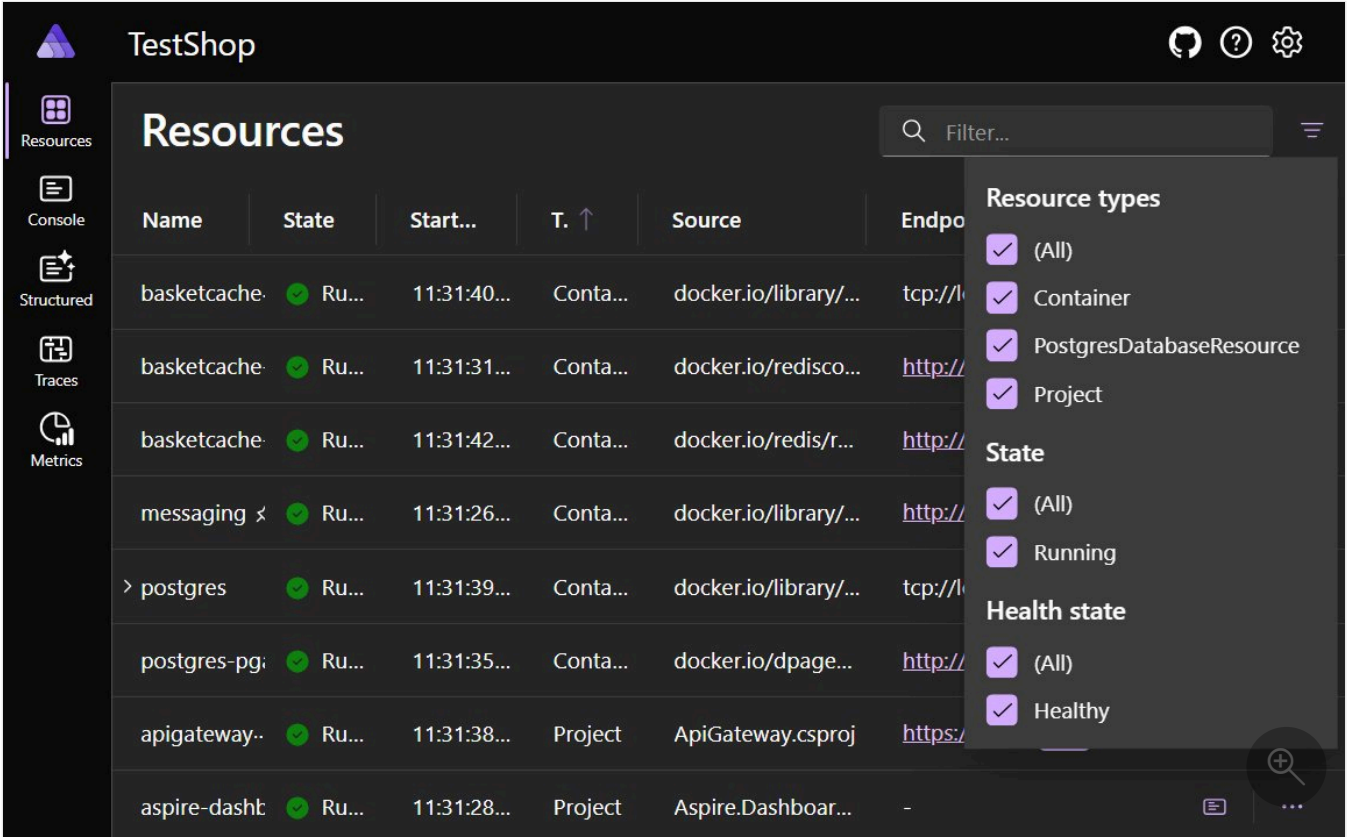
💡 Tip

If you're interested in contributing to .NET Aspire, look for issues labeled with [good first issue](#) and follow the [contributor guide](#).

1234

New filtering

You can now filter what you see in the **Resource** page by **Resource type**, **State**, and **Health state**. Consider the following screen capture, which demonstrates the addition of the filter options in the dashboard:



More resource details

When you select a resource in the dashboard, the details pane now displays new data points, including **References**, **Back references**, and **Volumes** with their mount types. This enhancement provides a clearer and more comprehensive view of your resources, improving the overall user experience by making relevant details more accessible.

The screenshot shows the TestShop dashboard. On the left, a sidebar contains navigation icons for Resources, Console, Structured, Traces, and Metrics. The main area is titled 'Resources' and displays a table of running services. A red arrow points from the 'Endpoints' column of the 'postgres-pgadmin' service to the 'References' section of the 'basketservice' project details. Another red arrow points from the 'Endpoints' column of the 'catalogdbapp' service to the 'Back references' section of the same project details.

Name	State	Endpoints	Acti...
catalogdb	Running	-	...
postgres-pgadmin	Running	http://localhost:56790	...
apigateway	Running	https://localho... +1	...
aspire-dashboard	Running	-	...
basket-service	Running	https://localho... +1	...
catalogdbapp	Running	https://localho... +1	...
catalog-service-ba	Running	https://localho... +1	...
catalog-service-de	Running	https://localho... +1	...
frontend	Running	https://localho... +1	...
orderprocessor...	Running	-	...

Project: basketservice

View console logs

References 2

Resource	Type	View
basketcache	Reference	View
messaging	Reference, WaitFor	View

Back references 2

Resource	Type	View
apigateway	Reference	View
frontend	Reference	View

For more information, see [.NET Aspire dashboard: Resources page](#).

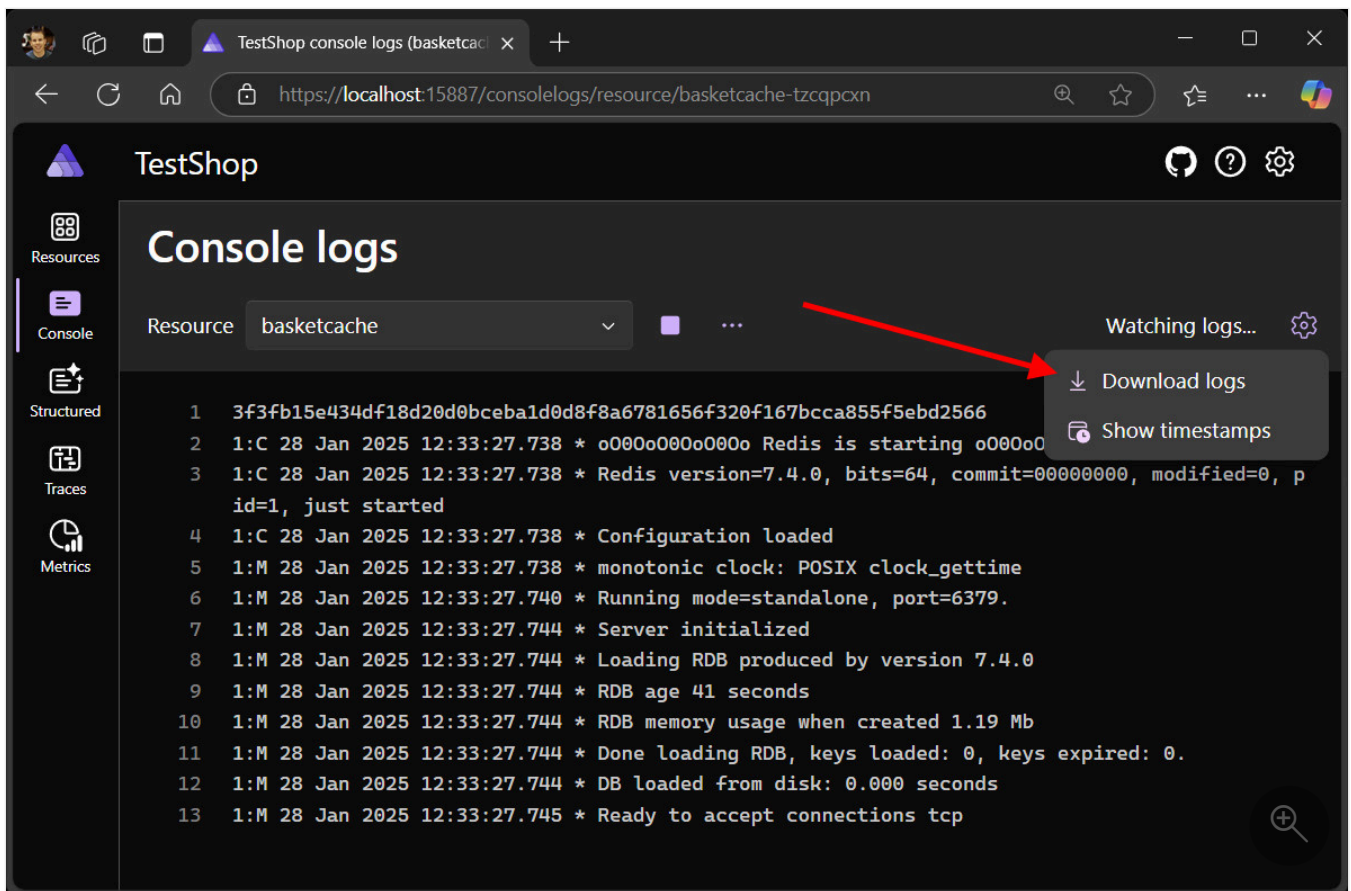
CORS support for custom local domains

You can now set the `ASPIRE_DASHBOARD_CORS_ALLOWED_ORIGINS` environment variable to allow the dashboard to receive telemetry from other browser apps, such as if you have resources running on custom localhost domains.

For more information, see [.NET Aspire app host: Dashboard configuration](#).

Flexibility with console logs

The console log page has two new options. You're now able to download your logs so you can view them in your own diagnostics tools. Plus, you can turn timestamps on or off to reduce visual clutter when needed.



For more information, see [.NET Aspire dashboard: Console logs page](#).

Various UX improvements

Several new features in .NET Aspire 9.1 enhance and streamline the following popular tasks:

-  Resource commands, such as **Start** and **Stop** buttons, are now available on the **Console logs** page.
-  Single selection to open in the *text visualizer*.
-  URLs within logs are now automatically clickable, with commas removed from endpoints.

Additionally, the  scroll position resets when switching between different resources—this helps to visually reset the current resource view.

For more details on the latest dashboard enhancements, check out [James Newton-King on Bluesky](#) , where he's been sharing new features daily.

Local development enhancements

In .NET Aspire 9.1, several improvements to streamline your local development experience were an emphasis. These enhancements are designed to provide greater flexibility, better

integration with Docker, and more efficient resource management. Here are some of the key updates:

Start resources on demand

You can now tell resources not to start with the rest of your app by using [WithExplicitStart](#) on the resource in your app host. Then, you can start it whenever you're ready from inside the dashboard.

For more information, see [Configure explicit resource start](#).

Better Docker integration

The `PublishAsDockerfile()` feature was introduced for all projects and executable resources. This enhancement allows for complete customization of the Docker container and Dockerfile used during the publish process.

While this API was available in previous versions, it couldn't be used with [ProjectResource](#) or [ExecutableResource](#) types.

Cleaning up Docker networks

In 9.1, we addressed a persistent issue where Docker networks created by .NET Aspire would remain active even after the application was stopped. This bug, tracked in [.NET Aspire GitHub issue #6504](#), is resolved. Now, Docker networks are properly cleaned up, ensuring a more efficient and tidy development environment.

Socket address issues fixed

Several users reported issues ([#6693](#), [#6704](#), [#7095](#)) with restarting the .NET Aspire app host, including reconciliation errors and "address already in use" messages.

This release introduces a more robust approach to managing socket addresses, ensuring only one instance of each address is used at a time. Additionally, improvements were made to ensure proper project restarts and resource releases, preventing hanging issues. These changes enhance the stability and reliability of the app host, especially during development and testing.

Integration updates

.NET Aspire continues to excel through its [integrations](#) with various platforms. This release includes numerous updates to existing integrations and details about ownership migrations,

enhancing the overall functionality and user experience.

Azure updates

This release also focused on improving various [Azure integrations](#):

New emulators

We're excited to bring new emulators for making local development easier. The following integrations got new emulators in this release:

- [Azure Service Bus](#)
- [Azure Cosmos DB Linux-based \(preview\)](#)
- [Azure SignalR](#)

C#

```
var serviceBus = builder.AddAzureServiceBus("servicebus")
                        .RunAsEmulator();

#pragma warning disable ASPIRECOSMOSDB001
var cosmosDb = builder.AddAzureCosmosDB("cosmosdb")
                      .RunAsPreviewEmulator();

var signalr = builder.AddAzureSignalR("signalr",
AzureSignalRServiceMode.Serverless)
                  .RunAsEmulator();
```

These new emulators work side-by-side with the existing emulators for:

- [Azure Storage](#)
- [Azure Event Hubs](#)
- [Azure Cosmos DB](#)

Cosmos DB

Along with support for the new emulator, Cosmos DB added the following features.

Support for Entra ID authentication by default

Previously, the Cosmos DB integration used access keys and a Key Vault secret to connect to the service. .NET Aspire 9.1 added support for using more secure authentication using managed identities by default. If you need to keep using access key authentication, you can get back to the previous behavior by calling [WithAccessKeyAuthentication](#).

Support for modeling Database and Containers in the app host

You can define a Cosmos DB database and containers in the app host and these resources are available when you run the application in both the emulator and in Azure. This allows you to define these resources up front and no longer need to create them from the application, which might not have permission to create them.

For example API usage to add database and containers, see the following related articles:

- [.NET Aspire Azure Cosmos DB integration](#)
- [.NET Aspire Cosmos DB Entity Framework Core integration](#)

Support for Cosmos DB-based triggers in Azure Functions

The [AzureCosmosDBResource](#) was modified to support consumption in Azure Functions applications that uses the Cosmos DB trigger. A Cosmos DB resource can be initialized and added as a reference to an Azure Functions resource with the following code:

C#

```
var cosmosDb = builder.AddAzureCosmosDB("cosmosdb")
                        .RunAsEmulator();
var database = cosmosDb.AddCosmosDatabase("mydatabase");
database.AddContainer("mycontainer", "/id");

var funcApp =
builder.AddAzureFunctionsProject<Projects.AzureFunctionsEndToEnd_Functions>
("funcapp")
    .WithReference(cosmosDb)
    .WaitFor(cosmosDb);
```

The resource can be used in the Azure Functions trigger as follows:

C#

```
public class MyCosmosDbTrigger(ILogger<MyCosmosDbTrigger> logger)
{
    [Function(nameof(MyCosmosDbTrigger))]
    public void Run([CosmosDBTrigger(
        databaseName: "mydatabase",
        containerName: "mycontainer",
        CreateLeaseContainerIfNotExists = true,
        Connection = "cosmosdb")] IReadOnlyList<Document> input)
    {
        logger.LogInformation(
            "C# cosmosdb trigger function processed: {Count} messages",
            input.Count);
    }
}
```

```
}  
}
```

For more information using Azure Functions with .NET Aspire, see [.NET Aspire Azure Functions integration \(Preview\)](#).

Service Bus and Event Hubs

Similar to Cosmos DB, the Service Bus and Event Hubs integrations now allow you to define Azure Service Bus queues, topics, subscriptions, and Azure Event Hubs instances and consumer groups directly in your app host code. This enhancement simplifies your application logic by enabling the creation and management of these resources outside the application itself.

For more information, see the following updated articles:

- [.NET Aspire Azure Service Bus integration](#)
- [.NET Aspire Azure Event Hubs integration](#)

Working with existing resources

There's consistent feedback about making it easier to connect to existing Azure resources in .NET Aspire. With 9.1, you can now easily connect to an existing Azure resource either directly by `string` name, or with [app model parameters](#) which can be changed at deployment time.

For example to connect to an Azure Service Bus account, we can use the following code:

```
C#  
  
var existingServiceBusName = builder.AddParameter("serviceBusName");  
var existingServiceBusResourceGroup =  
builder.AddParameter("serviceBusResourceGroup");  
  
var serviceBus = builder.AddAzureServiceBus("messaging")  
                    .AsExisting(existingServiceBusName,  
existingServiceBusResourceGroup);
```

The preceding code reads the name and resource group from the parameters, and connects to the existing resource when the application is run or deployed. For more information, see [use existing Azure resources](#).

Azure Container Apps

Experimental support for configuring custom domains in Azure Container Apps (ACA) was added. For example:

C#

```
#pragma warning disable ASPIREACADOMAINS001

var customDomain = builder.AddParameter("customDomain");
var certificateName = builder.AddParameter("certificateName");

builder.AddProject<Projects.AzureContainerApps_ApiService>("api")
    .WithExternalHttpEndpoints()
    .PublishAsAzureContainerApp((infra, app) =>
    {
        app.ConfigureCustomDomain(customDomain, certificateName);
    });
```

For more information, see [.NET Aspire diagnostics overview](#).

✚ Even more integration updates

- OpenAI now supports the  [Microsoft.Extensions.AI](#)  NuGet package.
- RabbitMQ updated to version 7, and MongoDB to version 3. These updates introduced breaking changes, leading to the release of new packages with version-specific suffixes. The original packages continue to use the previous versions, while the new packages are as follows:
 -  [Aspire.RabbitMQ.Client.v7](#)  NuGet package. For more information, see the [.NET Aspire RabbitMQ client integration](#) documentation.
 -  [Aspire.MongoDB.Driver.v3](#)  NuGet package. For more information, see the [.NET Aspire MongoDB client integration](#) documentation.
- Dapr migrated to the [CommunityToolkit](#)  to facilitate faster innovation.
- Numerous other integrations received updates, fixes, and new features. For detailed information, refer to our [GitHub release notes](#) .

The  [Aspire.Hosting.AWS](#)  NuGet package and source code migrated under [Amazon Web Services \(AWS\) ownership](#) . This migration happened as part of .NET Aspire 9.0, we're just restating that change here.

Testing in .NET Aspire

.NET Aspire 9.1 simplifies writing cross-functional integration tests with a robust approach. The app host allows you to create, evaluate, and manage containerized environments seamlessly within a test run. This functionality supports popular testing frameworks like xUnit, NUnit, and MSTest, enhancing your testing capabilities and efficiency.

Now, you're able to disable port randomization or enable the [dashboard](#). For more information, see [.NET Aspire testing overview](#). Additionally, you can now [Pass arguments to your app host](#).

Some of these enhancements were introduced as a result of stability issues that were reported, such as [.NET Aspire GitHub issue #6678](#) —where some resources failed to start do to "address in use" errors.

Deployment

Significant improvements to the Azure Container Apps (ACA) deployment process are included in .NET Aspire 9.1, enhancing both the `azd` CLI and app host options. One of the most requested features—support for deploying `npm` applications to ACA—is now implemented. This new capability allows `npm` apps to be deployed to ACA just like other resources, streamlining the deployment process and providing greater flexibility for developers.

We recognize there's more work to be done in the area of deployment. Future releases will continue to address these opportunities for improvement. For more information on deploying .NET Aspire to ACA, see [Deploy a .NET Aspire project to Azure Container Apps](#).

Breaking changes

.NET Aspire is moving quickly, and with that comes breaking changes. Breaking are categorized as either:

- **Binary incompatible:** The assembly version has changed, and you need to recompile your code.
- **Source incompatible:** The source code has changed, and you need to change your code.
- **Behavioral change:** The code behaves differently, and you need to change your code.

Typically APIs are decorated with the [ObsoleteAttribute](#) giving you a warning when you compile, and an opportunity to adjust your code. For an overview of breaking changes in .NET Aspire 9.1, see [Breaking changes in .NET Aspire 9.1](#).

Upgrade today

Follow the directions outlined in the [Upgrade to .NET Aspire 9.1](#) section to make the switch to 9.1 and take advantage of all these new features today! As always, we're listening for your feedback on [GitHub](#) —and looking out for what you want to see in 9.2 ☺.

For a complete list of issues addressed in this release, see [.NET Aspire GitHub repository—9.1 milestone](#) .

What's new in .NET Aspire 9.0

Article • 11/13/2024

 .NET Aspire 9.0 is the next major general availability (GA) release of .NET Aspire; it supports *both*:

- .NET 8.0 Long Term Support (LTS) *or*
- .NET 9.0 Standard Term Support (STS).

! Note

You're able to use .NET Aspire 9.0 with either .NET 8 or .NET 9!

This release addresses some of the most highly requested features and pain points from the community. The best features are community-driven! To join the community on, visit us on [Discord](#) to chat with team members and collaborate with us on [GitHub](#).

For more information on the official .NET version and .NET Aspire version support, see:

- [.NET support policy](#): Definitions for LTS and STS.
- [.NET Aspire support policy](#): Important unique product life cycle details.

Upgrade to .NET Aspire 9

To upgrade from earlier versions of .NET Aspire to .NET Aspire 9, follow the instructions in the official [Upgrade to .NET Aspire 9](#) guide. The guide provides detailed instructions on how to upgrade your existing .NET Aspire solutions to .NET Aspire 9. Regardless of you're doing it manually, or using the Upgrade Assistant, the guide makes short work of the process.

Tooling improvements

.NET Aspire 9 makes it simpler to configure your environment to develop .NET Aspire applications. You no longer need a .NET workload. Instead, you install the new [.NET Aspire SDK](#) into the app host project of your .NET Aspire solutions. For more information, see [.NET Aspire setup and tooling](#).

Templates have moved

.NET Aspire 9 is moving the contents that used to be installed via the workload into separate NuGet packages. This includes the templates for creating new .NET Aspire projects and solutions. These templates are installed using the [dotnet new install](#) command. These can be installed by running the following command:

```
.NET CLI
```

```
dotnet new install Aspire.ProjectTemplates::9.0.0
```

Tip

If you already have the .NET Aspire workload installed, you need to pass the `--force` flag to overwrite the existing templates. Feel free to uninstall the .NET Aspire workload.

For more information, see [.NET Aspire templates](#).

Dashboard UX enhancements and new interactivity features

The [.NET Aspire dashboard](#) continues to improve with each release.

Manage resource lifecycle

The most requested feature for the dashboard is to manage the life-cycles of your orchestrated named resources. Specifically, the ability to stop, start, and restart resources. This feature works for projects, containers, and executables. It enables restarting individual resources without having to restart the entire app host. For project resources, when the debugger is attached, it's reattached on restart. For more information, see [.NET Aspire dashboard: Stop or Start a resource](#).

Mobile and responsive support

The .NET Aspire dashboard is now mobile-friendly, responsively adapting to a wide range of screen sizes and enabling on-the-go management of deployed .NET Aspire applications. Other accessibility improvements were made, including the display of settings and content overflow on mobile.

Sensitive properties, volumes, and health checks in resource details

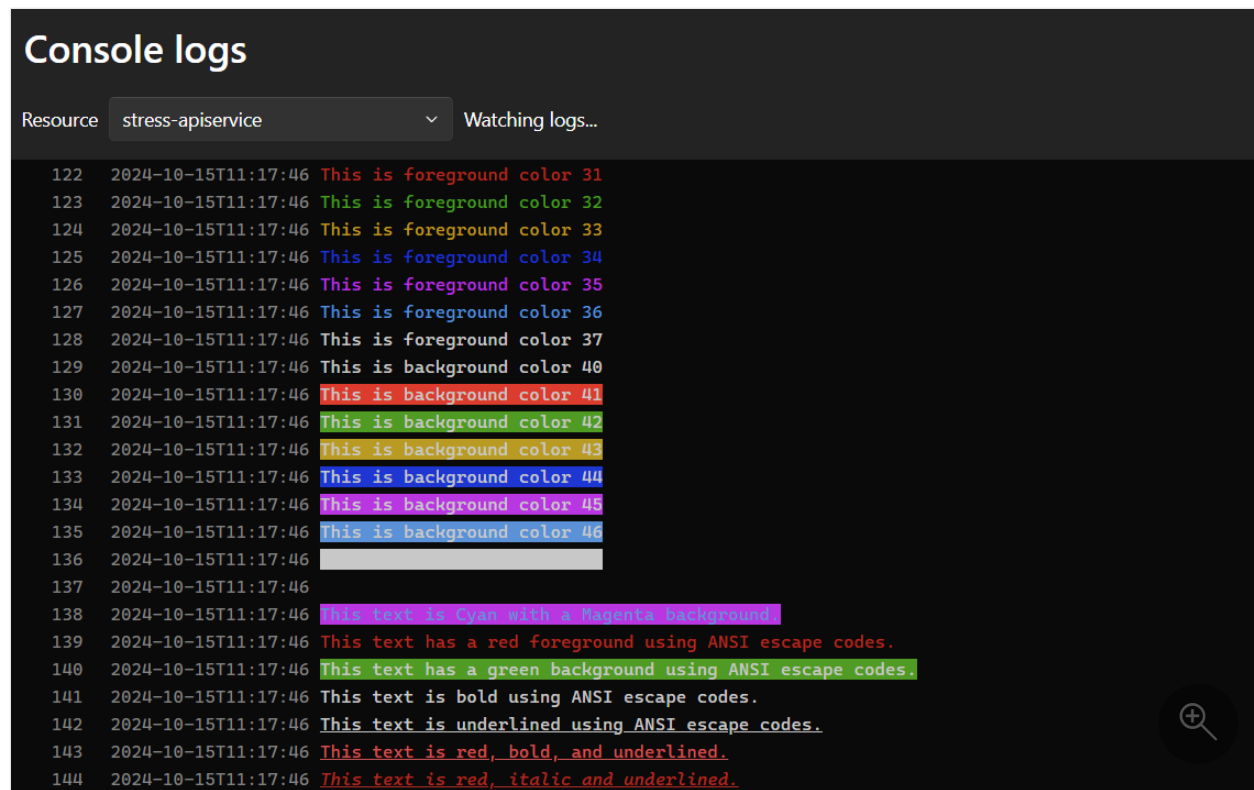
The display of resource details contains several improvements:

- Properties can be marked as sensitive, automatically masking them in the dashboard UI. This security feature helps to avoid accidentally disclosing keys or passwords when screen sharing the dashboard with other people. For example, container arguments could pass sensitive information and so are masked by default.
- Configured container volumes are listed in resource details.
- .NET Aspire 9 adds support for health checks. Detailed information about these checks can now be viewed in the resource details pane, showing why a resource might be marked as unhealthy or degraded. Find out more about health checks [here](#).

Colorful console log

[ANSI escape codes](#) [↗](#) format text in terminals by controlling colors (foreground and background) and styles like bold, underline, and italics. Previously, the dashboard's console logs page could only render one ANSI escape code at a time, failing when multiple codes were combined. For example, it could display red text, but not text that was both red and bold.

A community contribution from [@mangeg](#) [↗](#) improved support for ANSI escape codes and removed this limitation.



Another improvement to console logs is hiding unsupported escape codes. Codes that aren't related to displaying text, such as positioning the cursor or communicating with the operating system don't make sense in this UI, and are hidden.

Telemetry user-centric additions

[Telemetry](#) remains a vital aspect of .NET Aspire. In .NET Aspire 9, many new features were introduced to the Telemetry service.

Improved telemetry filtering

Traces can be filtered with attribute values. For example, if you only want to view traces for one endpoint in your app, the `http.route` attribute on HTTP requests can be filtered to a specified value.

Telemetry filtering also supports autocomplete of existing values. The **Add filter** dialog provides a combo box for selecting from values that dashboard has available. This feature makes it much easier to filter to real data and helps avoid typos by entered a value yourself.

For more information, see [.NET Aspire dashboard: Filter traces](#).

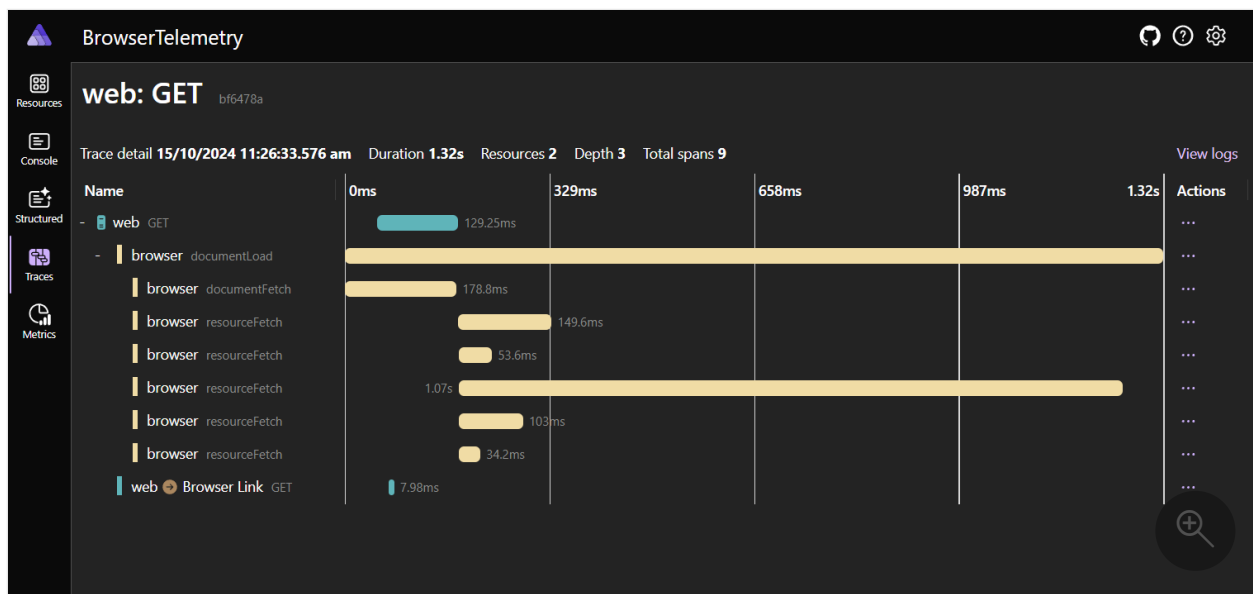
Combine telemetry from multiple resources

When a resource has multiple replicas, you can now filter telemetry to view data from all instances at once. Select the parent resource, labeled `(application)`. For more information, see [.NET Aspire dashboard: Combine telemetry from multiple resources](#).

Browser telemetry support

The dashboard supports OpenTelemetry Protocol (OTLP) over HTTP and cross-origin resource sharing (CORS). These features unlock the ability to send OpenTelemetry from browser apps to the .NET Aspire dashboard.

For example, a browser-based single page app (SPA) can configure the [JavaScript OpenTelemetry SDK](#) to send structured logs, traces, and metrics created in the browser to the dashboard. Browser telemetry is displayed alongside server telemetry.



For more information on configuring browser telemetry, see [Enable browser telemetry](#) documentation.

App Host (Orchestration)

The [.NET Aspire app host](#) is one of the **most important** features of .NET Aspire. In .NET Aspire 9, several new features were added specific to the app host.

Waiting for dependencies

If you've been following along with .NET Aspire, you already know that your app host project is where you define your app model. You create a distributed application builder, add and configure resources, and express their dependencies. Now, you can specify that a resource should *wait* for another resource before starting. This can help avoid

connection errors during startup by only starting resources when their dependencies are "ready."

C#

```
var builder = DistributedApplication.CreateBuilder(args);

var rabbit = builder.AddRabbitMQ("rabbit");

builder.AddProject<Projects.WebApplication1>("api")
    .WithReference(rabbit)
    .WaitFor(rabbit); // Don't start "api" until "rabbit" is ready...

builder.Build().Run();
```

When the app host starts, it waits for the `rabbit` resource to be ready before starting the `api` resource.

There are two methods exposed to wait for a resource:

- [WaitFor](#): Wait for a resource to be ready before starting another resource.
- [WaitForCompletion](#): Wait for a resource to complete before starting another resource.

For more information, see [.NET Aspire app host: Waiting for resources](#).

Resource health checks

The `WaitFor` API uses standard [.NET health checks](#) to determine if a resource is ready. But what does "a resource being ready" mean? The best part is, that's configurable by the consumer beyond their default values.

When a resource doesn't expose any health checks (no health checks registered in the app), the app host waits for the resource to be in the [Running](#) state before starting the dependent resource.

For resources that expose HTTP endpoints, you can easily add a health check that polls a specific path for an HTTP 200 response.

C#

```
var builder = DistributedApplication.CreateBuilder(args);

var catalogApi = builder.AddContainer("catalog-api", "catalog-api")
    .WithHttpEndpoint(targetPort: 8080)
    .WithHttpHealthCheck("/health");
```

```
builder.AddProject<Projects.WebApplication1>("store")
    .WithReference(catalogApi.GetEndpoint("http"))
    .WaitFor(catalogApi);

builder.Build().Run();
```

The preceding example adds a health check to the `catalog-api` resource. The app host waits for the health check to return a healthy status before starting the `store` resource. It determines that the resource is ready when the `/health` endpoint returns an HTTP 200 status code.

While `store` is waiting for `catalog-api` to become healthy, the resources in the dashboard appear as:

Type	Name	State
Project	catalog-api	 Running (Unhealthy)
Project	store	 Waiting 

The app host's health check mechanism builds upon the [IHealthChecksBuilder](#) implementation from the [Microsoft.Extensions.Diagnostics.HealthChecks](#) namespace.

Health checks report data, which is displayed in the dashboard:

Health checks 2 ^		
Name	State	Details
catalog-api_http_/healthz_200_check	 Unhealthy	Discover endpoint #0 is not responding...
service-ready	 Unhealthy	Service is starting 

Creating a custom health check is straightforward. Start by defining the health check, then associate its name with any resources it applies to.

```
C#

var builder = DistributedApplication.CreateBuilder(args);

var healthyAfter = DateTime.Now.AddSeconds(20);
```

```
builder.Services.AddHealthChecks().AddCheck(
    "delay20secs",
    () => DateTime.Now > healthyAfter
        ? HealthCheckResult.Healthy()
        : HealthCheckResult.Unhealthy()
);

var cache = builder.AddRedis("cache")
    .WithHealthCheck("delay20secs");

builder.AddProject<Projects.MyApp>("myapp")
    .WithReference(cache)
    .WaitFor(cache);
```

The preceding example adds a health check to the `cache` resource, which reports it as unhealthy for the first 20 seconds after the app host starts. So, the `myapp` resource waits for 20 seconds before starting, ensuring the `cache` resource is healthy.

The [AddCheck](#) and [WithHealthCheck](#) methods provide a simple mechanism to create health checks and associate them with specific resources.

Persistent containers

The app host now supports *persistent* containers. Persistent containers deviate from the [typical container life cycle of .NET Aspire orchestrated apps](#). While they're *created* and *started* (when not already available) by the .NET Aspire orchestrator, they're not destroyed by .NET Aspire.

This is useful when you want to keep the container running even after the app host has stopped.

Important

To delete these containers, you must manually stop them using the container runtime.

To define an `IResourceBuilder<ContainerResource>` with a persistent lifetime, call the [WithLifetime](#) method and pass in [ContainerLifetime.Persistent](#):

```
C#

var builder = DistributedApplication.CreateBuilder(args);

var queue = builder.AddRabbitMQ("rabbit")
    .WithLifetime(ContainerLifetime.Persistent);
```



```
builder.AddProject<Projects.WebApplication1>("api")
    .WithReference(queue)
    .WaitFor(queue);

builder.Build().Run();
```

The dashboard shows persistent containers with a pin icon:

Type	Name	State
Container	rabbit  	 Running
Project	api	 Running 

After the app host is stopped, the container will continue to run:

	Name	Image	Status	Port(s)	CPU (%)	Last started	Actions
	 rabbit-c7d291cf 71f69ace9107 	library/rabbitmq:3.13	Running		0.25%	5 minutes ago	   

The container persistence mechanism attempts to identify when you might wish to recreate the container. For example, if the environment for the container changes, then the container is restarted so that you don't need to manually stop the container if the input configuration for the resource has changed.

Resource commands

The app host supports adding custom commands to resources. This is useful when you want to add custom functionality that is not natively supported by the app host. There's likely many opportunities where exposing custom extension methods on resources will be useful. The [.NET Aspire Community Toolkit](#) might be a good place to share these extensions.

When you define a custom command, it's available in the dashboard as a user experience feature.

Important

These .NET Aspire dashboard commands are only available when running the dashboard locally. They're not available when running the dashboard in Azure

For more information on creating custom resource commands, see [How-to: Create custom resource commands in .NET Aspire](#).

Container networking

The app host now adds all containers to a common network named `default-aspire-network`. This is useful when you want to communicate between containers without going through the host network. This also makes it easier to migrate from docker compose to the app host, as containers can communicate with each other using the container name.

Eventing model

The eventing model allows developers to hook into the lifecycle of the application and resources. This is useful for running custom code at specific points in the application lifecycle. There are various ways to subscribe to events, including global events and per-resource events.

Global events:

- [BeforeStartEvent](#): An event that is triggered before the application starts. This is the last place that changes to the app model are observed. This runs in both "Run" and "Publish" modes. This is a blocking event, meaning that the application doesn't start until all handlers have completed.
- [AfterResourcesCreatedEvent](#): An event that is triggered after the resources are created. This runs in Run mode only.
- [AfterEndpointsAllocatedEvent](#): An event that is triggered after the endpoints are allocated for all resources. This runs in Run mode only.

The global events are analogous to the app host life cycle events. For more information, see [App host life cycles](#).

Per-resource events:

- [BeforeResourceStartedEvent](#): An event that is triggered before a single resource starts. This runs in Run mode only. This is a blocking event, meaning that the resource doesn't start until all handlers complete.
- [ConnectionStringAvailableEvent](#): An event that is triggered when a connection string is available for a resource. This runs in Run mode only.

- [ResourceReadyEvent](#): An event that is triggered when a resource is ready to be used. This runs in Run mode only.

For more information, see [Eventing in .NET Aspire](#).

Integrations

.NET Aspire continues to add integrations that make it easy to get started with your favorite services and tools. For more information, see [.NET Aspire integrations overview](#).

Redis Insight

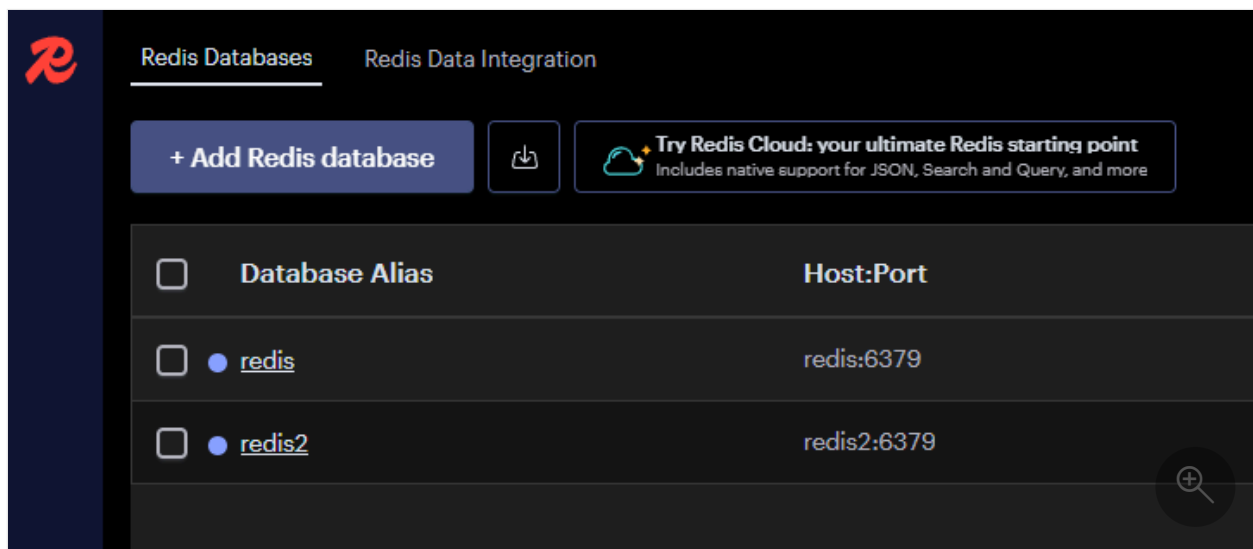
Support for [Redis Insights](#) is available on a Redis resource:

C#

```
var builder = DistributedApplication.CreateBuilder(args);

builder.AddRedis("redis")
    .WithRedisInsight(); // Starts a Redis Insight container image
                        // that is pre-configured to work with the
                        // Redis instance.
```

The [WithRedisInsight](#) extension method can be applied to multiple Redis resources and they'll each be visible on the Redis Insight dashboard.



For more information, see [Add Redis resource with Redis Insights](#).

OpenAI (Preview)

Starting with .NET Aspire 9, an additional OpenAI integration is available which allows to use the latest official OpenAI dotnet library directly. The client integration registers the [OpenAIClient](#) as a singleton service in the service collection. The client can be used to interact with the OpenAI REST API.

-  [Aspire.OpenAI \(Preview\)](#)

Moreover, the already available [.NET Aspire Azure OpenAI integration](#) was improved to provide a flexible way to configure an `OpenAIClient` for either an Azure AI OpenAI service or a dedicated OpenAI REST API one with the new [AddOpenAIClientFromConfiguration\(IHostApplicationBuilder, String\)](#) builder method. The following example detects if the connection string is for an Azure AI OpenAI service and registers the most appropriate `OpenAIClient` instance automatically.

C#

```
builder.AddOpenAIClientFromConfiguration("openai");
```

For instance, if the `openai` connection looked like

`Endpoint=https://{account}.azure.com;Key={key}`; it would guess it can register an Azure AI OpenAI client because of the domain name. Otherwise a common `OpenAIClient` would be used.

Read [Azure-agnostic client resolution](#) for more details.

MongoDB

Added support for specifying the MongoDB username and password when using the [AddMongoDB\(IDistributedApplicationBuilder, String, Nullable<Int32>, IResourceBuilder<ParameterResource>, IResourceBuilder<ParameterResource>\)](#) extension method. If not specified, a random username and password is generated but can be manually specified using parameter resources.

C#

```
var builder = DistributedApplication.CreateBuilder(args);

var username = builder.AddParameter("mongousername");
var password = builder.AddParameter("mongopassword", secret: true);

var db = builder.AddMongo("db", username, password);
```

Important Azure improvements

The following sections describe Azure improvements added in .NET Aspire 9. For a complete listing of all the breaking changes, see [Breaking changes in .NET Aspire 9](#).

Azure resource customization

In .NET Aspire 8, customizing Azure resources were marked experimental because the underlying `Azure.Provisioning` libraries were new and gathering feedback before they could be marked stable. In .NET Aspire 9 these APIs were updated and removes the experimental attribute.

Azure Resource naming breaking change

As part of the update to the [Azure.Provisioning](#) libraries, the default naming scheme for Azure resources was updated with better support for various naming policies. However, this update resulted in a change to how resources are named. The new naming policy might result in the existing Azure resources being abandoned and new Azure resources being created, after updating your .NET Aspire application from 8 to 9. To keep using the same naming policies from .NET Aspire 8, you can add the following code to your AppHost *Program.cs*:

C#

```
var builder = DistributedApplication.CreateBuilder(args);

builder.Services.Configure<AzureProvisioningOptions>(options =>
{
    options.ProvisioningBuildOptions.InfrastructureResolvers.Insert(0, new
    AspireV8ResourceNamePropertyResolver());
});
```

Azure SQL, PostgreSQL, and Redis Update

Azure SQL, PostgreSQL, and Redis resources are different than other Azure resources because there are local container resources for these technologies. In .NET Aspire 8, in order to create these Azure resources you needed to start with a local container resource and then either "As" or "PublishAs" it to an Azure resource. This design introduced problems and didn't fit with other APIs.

For example, you might have this code in .NET Aspire 8:

C#

```

var builder = DistributedApplication.CreateBuilder(args);

var sql = builder.AddSqlServer("sql")
    .PublishAsAzureSqlDatabase();

var pgsql = builder.AddPostgres("pgsql")
    .PublishAsAzurePostgresFlexibleServer();

var cache = builder.AddRedis("cache")
    .PublishAsAzureSqlDatabase();

```

In .NET Aspire 9 these APIs were marked as obsolete and a new API pattern implemented:

C#

```

var builder = DistributedApplication.CreateBuilder(args);

var sql = builder.AddAzureSqlServer("sql")
    .RunAsContainer();

var pgsql = builder.AddAzurePostgresFlexibleServer("pgsql")
    .RunAsContainer();

var cache = builder.AddAzureRedis("cache")
    .RunAsContainer();

```

Microsoft Entra ID by default

In order to make .NET Aspire applications more secure, Azure Database for PostgreSQL and Azure Cache for Redis resources were updated to use Microsoft Entra ID by default. This requires changes to applications that need to connect to these resources. See the following for updating applications to use Microsoft Entra ID to connect to these resources:

- [Azure Database for PostgreSQL](#) ↗
- [Azure Cache for Redis](#) ↗

The following examples demonstrate how to configure your application to connect to the Azure resources using Microsoft Entra ID:

- [.NET Aspire: Azure PostgreSQL hosting integration.](#)
- [.NET Aspire: Azure Redis hosting integration.](#)

If you need to use password or access key authentication (not recommended), you can opt-in with the following code:

C#

```
var builder = DistributedApplication.CreateBuilder(args);

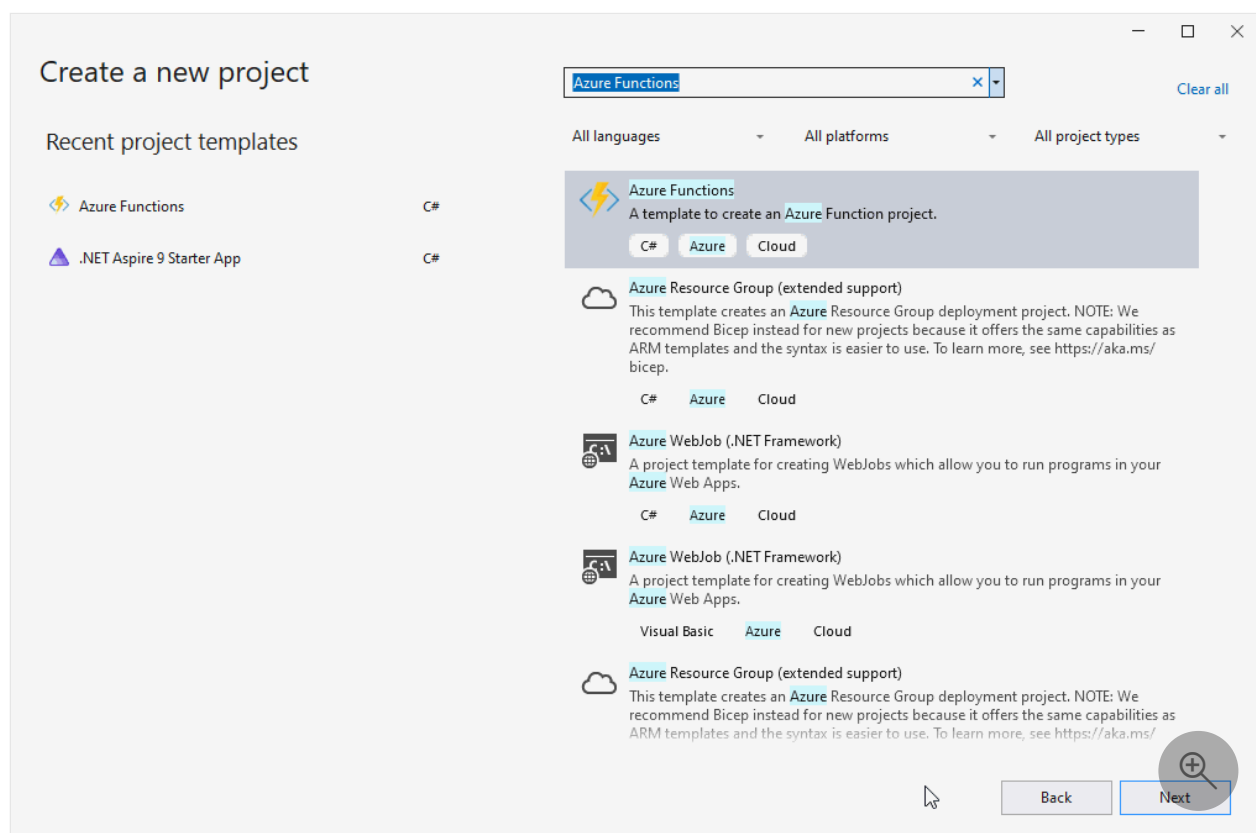
var pgsql = builder.AddAzurePostgresFlexibleServer("pgsql")
    .WithPasswordAuthentication();

var cache = builder.AddAzureRedis("cache")
    .WithAccessKeyAuthentication();
```

Support for Azure Functions (Preview)

Support for [Azure Functions](#) is one of the most widely requested features on the .NET Aspire issue tracker and we're excited to introduce preview support for it in this release. To demonstrate this support, let's use .NET Aspire to create and deploy a webhook.

To get started, create a new Azure Functions project using the **Visual Studio New Project** dialog. When prompted, select the **Enlist in Aspire orchestration** checkbox when creating the project.



In the app host project, observe that there's a `PackageReference` to the new  [Aspire.Hosting.Azure.Functions](#) NuGet package:

XML

```
<ItemGroup>
  <PackageReference Include="Aspire.Hosting.AppHost" Version="9.0.0" />
  <PackageReference Include="Aspire.Hosting.Azure.Functions"
Version="9.0.0" />
</ItemGroup>
```

This package provides an `AddAzureFunctionsProject<TProject>` (`IDistributedApplicationBuilder, String`) API that can be invoked in the app host to configure Azure Functions projects within an .NET Aspire host:

C#

```
var builder = DistributedApplication.CreateBuilder(args);

builder.AddAzureFunctionsProject<Projects.PigLatinApp>("piglatingapp");

builder.Build().Run();
```

In this example, the webhook is responsible for translating an input string into Pig Latin. Update the contents of our trigger with the following code:

C#

```
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Azure.Functions.Worker;
using Microsoft.Extensions.Logging;
using System.Text;
using FromBodyAttribute =
Microsoft.Azure.Functions.Worker.Http.FromBodyAttribute;

namespace PigLatinApp;

public class Function1(ILogger<Function1> logger)
{
    public record InputText(string Value);
    public record PigLatinText(string Value);

    [Function("Function1")]
    public IActionResult Run(
        [HttpTrigger(AuthorizationLevel.Anonymous, "post")] HttpRequest req,
        [FromBody] InputText inputText)
    {
        logger.LogInformation("C# HTTP trigger function processed a request.");

        var result = TranslateToPigLatin(inputText.Value);

        return new OkObjectResult(new PigLatinText(result));
    }
}
```



```

private static string TranslateToPigLatin(string input)
{
    if (string.IsNullOrEmpty(input))
    {
        return input;
    }

    var words = input.Split(' ');
    StringBuilder pigLatin = new();

    foreach (string word in words)
    {
        if (IsVowel(word[0]))
        {
            pigLatin.Append(word + "yay ");
        }
        else
        {
            int vowelIndex = FindFirstVowelIndex(word);
            if (vowelIndex is -1)
            {
                pigLatin.Append(word + "ay ");
            }
            else
            {
                pigLatin.Append(
                    word.Substring(vowelIndex) + word.Substring(0,
vowelIndex) + "ay ");
            }
        }
    }

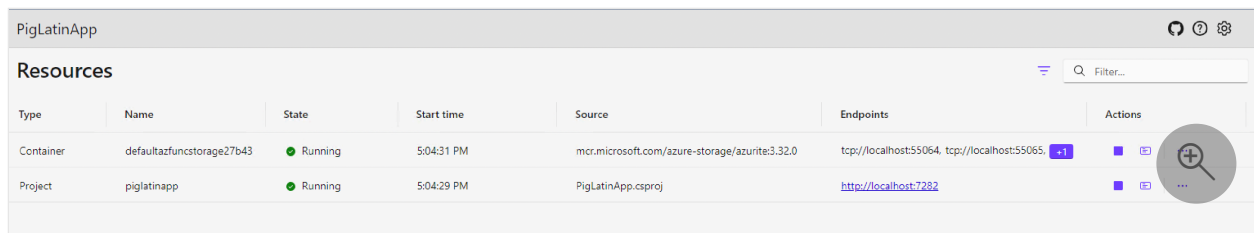
    return pigLatin.ToString().Trim();
}

private static int FindFirstVowelIndex(string word)
{
    for (var i = 0; i < word.Length; i++)
    {
        if (IsVowel(word[i]))
        {
            return i;
        }
    }
    return -1;
}

private static bool IsVowel(char c) =>
    char.ToLower(c) is 'a' or 'e' or 'i' or 'o' or 'u';
}

```

Set a breakpoint on the first `logger.LogInformation` line of the `Run` method and press `F5` to start the Functions host. Once the .NET Aspire dashboard launches, you observe the following:



The screenshot shows the .NET Aspire dashboard for a project named 'PigLatinApp'. It displays a table of resources with columns: Type, Name, State, Start time, Source, Endpoints, and Actions. Two resources are listed: a Container named 'defaultazfuncstorage27b43' and a Project named 'piglatinapp', both in a 'Running' state. The container's source is 'mcr.microsoft.com/azure-storage/azurite:3.32.0' and its endpoints are 'tcp://localhost:55064' and 'tcp://localhost:55065'. The project's source is 'PigLatinApp.csproj' and its endpoint is 'http://localhost:7282'.

Type	Name	State	Start time	Source	Endpoints	Actions
Container	defaultazfuncstorage27b43	Running	5:04:31 PM	mcr.microsoft.com/azure-storage/azurite:3.32.0	tcp://localhost:55064, tcp://localhost:55065	[Stop] [Refresh] [Add]
Project	piglatinapp	Running	5:04:29 PM	PigLatinApp.csproj	http://localhost:7282	[Stop] [Refresh] [Add]

.NET Aspire has:

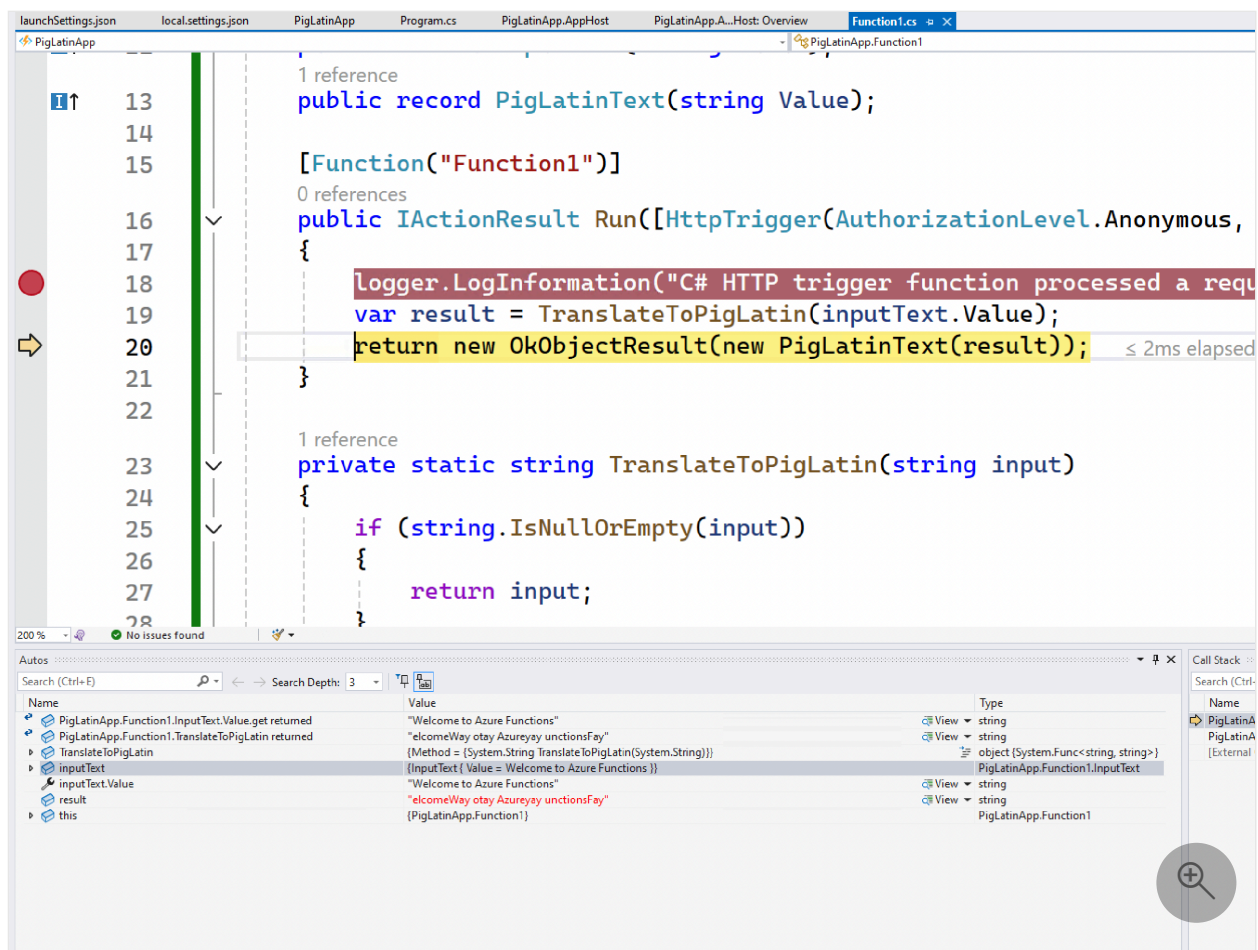
- Configured an emulated Azure Storage resource to be used for bookkeeping by the host.
- Launched the Functions host locally with the target as the Functions project registered.
- Wired the port defined in *launchSettings.json* of the functions project for listening.

Use your favorite HTTP client of choice to send a request to the trigger and observe the inputs bound from the request body in the debugger.

Windows

PowerShell

```
curl --request POST `
  --url http://localhost:7282/api/Function1 `
  --header 'Content-Type: application/json' `
  --data '{
    "value": "Welcome to Azure Functions"
  }'
```



Now you're ready to deploy our application to Azure Container Apps (ACA). Deployment currently depends on preview builds of Azure Functions Worker and Worker SDK packages. If necessary, upgrade the versions referenced in the Functions project:

XML

```

<ItemGroup>
  <PackageReference Include="Microsoft.Azure.Functions.Worker"
    Version="2.0.0-preview2" />
  <PackageReference Include="Microsoft.Azure.Functions.Worker.Sdk"
    Version="2.0.0-preview2" />
</ItemGroup>

```

You also need to expose a public endpoint for our Azure Functions project so that requests can be sent to our HTTP trigger:

C#

```

builder.AddAzureFunctionsProject<Projects.PigLatinApp>("piglatinapp")
    .WithExternalHttpEndpoints();

```

To deploy the application with the [azd CLI](#), you need get the latest version first. To install the latest version, you see a warning if your version is out of date. Follow the instructions to update to the latest version.

After it's installed, navigate to the folder containing the app host project and run `azd init`:

Azure Developer CLI

```
$ azd init
```

Initializing an app to run on Azure (azd init)

? How do you want to initialize your app? Use code in the current directory

(✓) Done: Scanning app code in current directory

Detected services:

.NET (Aspire)

Detected in: ./PigLatinApp/PigLatinApp.AppHost/PigLatinApp.AppHost.csproj

azd will generate the files necessary to host your app on Azure using Azure Container Apps.

? Select an option Confirm and continue initializing my app

? Enter a new environment name: azfunc-piglatin

Generating files to run your app on Azure:

(✓) Done: Generating ./azure.yaml

(✓) Done: Generating ./next-steps.md

SUCCESS: Your app is ready for the cloud!

Then, deploy the application by running `azd up`:

Azure Developer CLI

```
$ azd up
```

? Select an Azure Subscription to use: 130. [redacted]

? Select an Azure location to use: 50. (US) West US 2 (westus2)

Packaging services (azd package)

Provisioning Azure resources (azd provision)

Provisioning Azure resources can take some time.

Subscription: [redacted]

Location: West US 2

You can view detailed progress in the Azure Portal:

[redacted]

(✓) Done: Resource group: rg-azfunc-piglatin (967ms)

```
(✓) Done: Container Registry: [redacted] (13.316s)
(✓) Done: Log Analytics workspace: [redacted] (16.467s)
(✓) Done: Container Apps Environment: [redacted] (1m35.531s)
(✓) Done: Storage account: [redacted] (21.37s)
```

Deploying services (azd deploy)

```
(✓) Done: Deploying service piglatinapp
- Endpoint: {{endpoint-url}}
```

Aspire Dashboard: {{dashboard-url}}

Finally, test your deployed Functions application using your favorite HTTP client:

Windows

PowerShell

```
curl --request POST `
  --url {{endpoint-url}}/api/Function1 `
  --header 'Content-Type: application/json' `
  --data '{
    "value": "Welcome to Azure Functions"
  }'
```

Support for Azure Functions in .NET Aspire is still in preview with support for a limited set of triggers including:

- [HTTP triggers](#)
- [Azure Storage Queue triggers](#)
- [Azure Storage Blob triggers](#)
- [Azure Service Bus triggers](#)
- [Azure Event Hubs triggers](#)

For more information, see the official [.NET Aspire Azure Functions integration \(Preview\)](#).

Customization of Azure Container Apps

One of the most requested features is the ability to customize the Azure Container Apps that the app host creates without touching Bicep. This is possible by using the

[PublishAsAzureContainerApp<T>\(IResourceBuilder<T>, Action<AzureResourceInfrastructure, ContainerApp>\)](#) and [PublishAsAzureContainerApp<T>\(IResourceBuilder<T>, Action<AzureResourceInfrastructure, ContainerApp>\)](#) APIs in the

`Aspire.Hosting.Azure.AppContainers` namespace. These methods customize the Azure Container App definition that the app host creates.

Add the package reference to your project file:

XML

```
<ItemGroup>
  <PackageReference Include="Aspire.Hosting.Azure.AppContainers"
    Version="9.0.0" />
</ItemGroup>
```

The following example demonstrates how to scale an Azure Container App to zero (0) replicas:

C#

```
var builder = DistributedApplication.CreateBuilder(args);

var db = builder.AddAzurePostgresFlexibleServer("pg")
    .RunAsContainer()
    .AddDatabase("db");

// Type is for evaluation purposes only and is subject to change or removal
// in future updates. Suppress this diagnostic to proceed.
#pragma warning disable AZPROVISION001

builder.AddProject<Projects.WebApplication1>("api")
    .WithReference(db)
    .PublishAsAzureContainerApp((module, containerApp) =>
    {
        // Scale to 0
        containerApp.Template.Value!.Scale.Value!.MinReplicas = 0;
    });

#pragma warning restore AZPROVISION001

builder.Build().Run();
```

The preceding code example defers generation of the Azure Container App definition to the app host. This allows you to customize the Azure Container App definition without needing to run `azd infra synth` and unsafely modifying the generated bicep files.

See also

- [.NET Aspire setup and tooling](#)
- [.NET Aspire SDK](#)

- [.NET Aspire templates](#)
- [.NET Aspire orchestration overview](#)
- [Eventing in .NET Aspire](#)
- [.NET Aspire dashboard overview](#)
- [Explore the .NET Aspire dashboard](#)

Breaking changes in .NET Aspire

Article • 04/10/2025

Use this reference section to find breaking changes that might apply to you if you're upgrading your app to a newer version of .NET Aspire. You can navigate the table of contents either by .NET Aspire version or by technology area.

If you're looking for breaking changes for .NET, see [Breaking changes in .NET](#).

GitHub issues and announcements

You can also view individual issues that detail the breaking changes introduced in .NET in the following GitHub repositories:

- For .NET Aspire issues tracking breaking changes, see [dotnet/aspire](#).
- For .NET Aspire pull requests introducing breaking changes, see [dotnet/aspire](#).

See also

- [API removal in .NET](#)
- [.NET Aspire Support Policy](#)
- [.NET runtime compatibility](#)

.NET Aspire docs: What's new for June 2025

07/02/2025

Welcome to what's new in the .NET Aspire docs for June 2025. This article lists some of the major changes to docs during this period.

Get started

Updated articles

- [Upgrade to .NET Aspire 9](#) - Add instructions to get rid of .NET Aspire 8 before upgrading to 9

Fundamentals

Updated articles

- [Standalone .NET Aspire dashboard](#) - Update container version for standalone dashboard examples

Database

Updated articles

- [.NET Aspire MongoDB database integration](#) - Documents the issue with deploying data volumes to ACA.
- [Apply Entity Framework Core migrations in .NET Aspire](#)
 - Altered EF Core Migrations tutorial to ensure that API waits for the migrations to complete
 - Add Microsoft.EntityFrameworkCore.Tools package guidance to EF Core migrations documentation
- [Seed data in a database using .NET Aspire](#) - Update seeding data article to Aspire 9.3

Messaging

Updated articles

- [.NET Aspire Azure Event Hubs integration](#)
 - Move the "Infrastructure as code" section of the Azure Overview article into a separate article
 - Recommend `AsExisting()` for Azure Integrations
- [.NET Aspire Azure Service Bus integration](#)
 - Move the "Infrastructure as code" section of the Azure Overview article into a separate article
 - Recommend `AsExisting()` for Azure Integrations
- [.NET Aspire Azure Web PubSub integration](#)
 - Move the "Infrastructure as code" section of the Azure Overview article into a separate article
 - Recommend `AsExisting()` for Azure Integrations

Security

Updated articles

- [.NET Aspire Azure Key Vault integration](#)
 - Move the "Infrastructure as code" section of the Azure Overview article into a separate article
 - Recommend `AsExisting()` for Azure Integrations

Deployment

New articles

- [Customize .NET Aspire Azure deployments](#)

Updated articles

- [Deploy a .NET Aspire project to Azure Container Apps](#) - Optimize ACA deployment docs workflow and content structure
- [Deploy a .NET Aspire project to Azure Container Apps using the Azure Developer CLI \(in-depth guide\)](#)
 - Optimize ACA deployment docs workflow and content structure
 - Update `aca-deployment-azd-in-depth.md`
- [Deploy a .NET Aspire project to Azure Container Apps using Visual Studio](#) - Optimize ACA deployment docs workflow and content structure

- [Tutorial: Deploy a .NET Aspire project using the Azure Developer CLI](#) - Add troubleshooting guidance for Azure DevOps pipeline "azd init" errors

Community contributors

The following people contributed to the .NET Aspire docs during this period. Thank you! Learn how to contribute by following the links under "Get involved" in the [what's new landing page](#).

- [Alirexaa](#)  - Alireza Baloochi Merged Pull Requests 1
- [Depechie](#)  - Glenn Versweyveld Merged Pull Requests 1
- [philip-reed](#)  - Phil Reed Merged Pull Requests 1

.NET Aspire docs: What's new for May 2025

06/02/2025

Welcome to what's new in the .NET Aspire docs for May 2025. This article lists some of the major changes to docs during this period.

Get started

Updated articles

- [Orchestrate Node.js apps in .NET Aspire](#) - Fix errors in seed-database-data.md

Fundamentals

Updated articles

- [.NET Aspire integrations overview](#) - Documentation for .NET Aspire RavenDB Integration
- [.NET Aspire setup and tooling](#)
 - Add Rancher Desktop details
 - Update setup-tooling.md to latest Aspire version
- [External parameters](#) - Add ReferenceExpression to the External Parameters article
- [Tutorial: Use the .NET Aspire dashboard with Python apps](#) - Update standalone-for-python.md

Database

New articles

- [.NET Aspire Azure SQL Entity Framework Core integration](#)
- [.NET Aspire Azure SQL integration](#)

Updated articles

- [.NET Aspire SQL Server Entity Framework Core integration](#) - Add Azure SQL integration articles
- [.NET Aspire SQL Server integration](#) - Add Azure SQL integration articles

- [Apply Entity Framework Core migrations in .NET Aspire](#) - Add explanatory notes to the EF Core Migrations tutorial.
- [Seed data in a database using .NET Aspire](#)
 - Fix errors in seed-database-data.md
 - Update Seed data in database article to 9.2

Community contributors

The following people contributed to the .NET Aspire docs during this period. Thank you! Learn how to contribute by following the links under "Get involved" in the [what's new landing page](#).

- [alistairmatthews](#)  - Alistair Matthews Merged Pull Requests 5
- [Alirexaa](#)  - Alireza Baloochi Merged Pull Requests 1
- [crisweber2600](#)  - Cris Weber Merged Pull Requests 1
- [shiranshalom](#)  - shiranshalom Merged Pull Requests 1

.NET Aspire docs: What's new for April 2025

Article • 05/02/2025

Welcome to what's new in the .NET Aspire docs for April 2025. This article lists some of the major changes to docs during this period.

Get started

Updated articles

- [Orchestrate Python apps in .NET Aspire](#)
 - Initial draft of diagnostic redesign
 - Fix the Python article
- [Quickstart: Build your first .NET Aspire solution](#) - Split content
- [Tutorial: Add .NET Aspire to an existing .NET app](#) - Update the add aspire to existing tutorial

Fundamentals

New articles

- [Orchestrate resources in .NET Aspire](#)

Updated articles

- [.NET Aspire and launch profiles](#) - Rename configs-replace prefix
- [.NET Aspire orchestration overview](#) - Split content
- [.NET Aspire service defaults](#) - Update service defaults and add filtering details
- [.NET Aspire setup and tooling](#) - Rename configs-replace prefix
- [.NET Aspire templates](#) - Add dev-cert details to templates content
- [Custom HTTP commands in .NET Aspire](#) - Split content
- [Dashboard configuration](#) - Rename configs-replace prefix
- [Enable browser telemetry](#) - Rename configs-replace prefix
- [Explore the .NET Aspire dashboard](#) - Update the Explore the .NET Aspire Dashboard article for v9.2.0
- [External parameters](#) - #3063 - Rename parameter name to not be 'value'
- [Orchestrate resources in .NET Aspire](#)

- Initial draft of diagnostic redesign
- Split content
- [Persist .NET Aspire project data using volumes or bind mounts](#) - Update the "Persist .NET Aspire project data using volumes" article
- [Security considerations for running the .NET Aspire dashboard](#) - Rename configs-replace prefix
- [Standalone .NET Aspire dashboard](#) - Rename configs-replace prefix

Database

Updated articles

- [.NET Aspire Milvus database integration](#) - Split content
- [.NET Aspire MongoDB database integration](#) - Split content
- [.NET Aspire Oracle Entity Framework Core integration](#) - Split content
- [.NET Aspire Qdrant integration](#) - Split content
- [Tutorial: Deploy a .NET Aspire project with a SQL Server Database to Azure](#) - Update the Tutorial: Deploy a .NET Aspire project with a SQL Server Database to Azure

Messaging

Updated articles

- [.NET Aspire Apache Kafka integration](#) - Split content
- [.NET Aspire Azure Event Hubs integration](#) - Removes obsolete API: Add breaking changes for `AzureOpenAIDeployment` obsolescence
- [.NET Aspire Azure Service Bus integration](#)
 - Split content
 - Removes obsolete API: Add breaking changes for `AzureOpenAIDeployment` obsolescence
- [.NET Aspire Azure Web PubSub integration](#) - Removes obsolete API: Add breaking changes for `AzureOpenAIDeployment` obsolescence
- [.NET Aspire NATS integration](#) - Split content
- [.NET Aspire RabbitMQ integration](#)
 - Split content
 - Clarify how to set credentials for the RabbitMQ management plugin

Caching

Updated articles

- [.NET Aspire Redis® distributed caching integration](#) - Correct keyed IDistributedCache registration and usage

Security

Updated articles

- [.NET Aspire Azure Key Vault integration](#) - Removes obsolete API: Add breaking changes for `AzureOpenAIDeployment` obsolescence

Deployment

Updated articles

- [Deploy a .NET Aspire project to Azure Container Apps using the Azure Developer CLI \(in-depth guide\)](#) - Terminate inline azd code snippet

Community contributors

The following people contributed to the .NET Aspire docs during this period. Thank you! Learn how to contribute by following the links under "Get involved" in the [what's new landing page](#).

- [alistairmatthews](#)  - Alistair Matthews Merged Pull Requests 4
- [a510](#)  - Ahmad Ibrahim Merged Pull Requests 1
- [dracan](#)  - Dan Clarke Merged Pull Requests 1
- [gabynevada](#)  - Elvis Nieves Merged Pull Requests 1
- [mip1983](#)  - Matthew Paul Merged Pull Requests 1
- [paulomorgado](#)  - Paulo Morgado Merged Pull Requests 1
- [ryanspain](#)  - Ryan Spain Merged Pull Requests 1